Chain of Responsibility Pattern

Resources

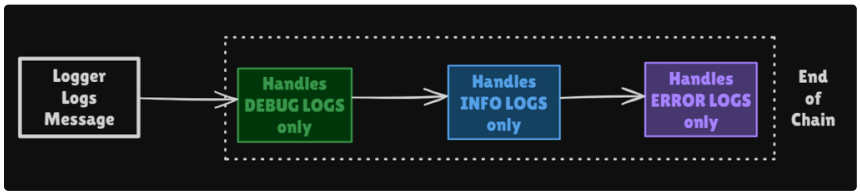
- Low Level Design from Basics to Advanced (Some Initial Videos are in Hindi, rest in English)
- 10. Design Logging System (Hindi) | Chain of Responsibility Design Pattern | System Design interview

Definition

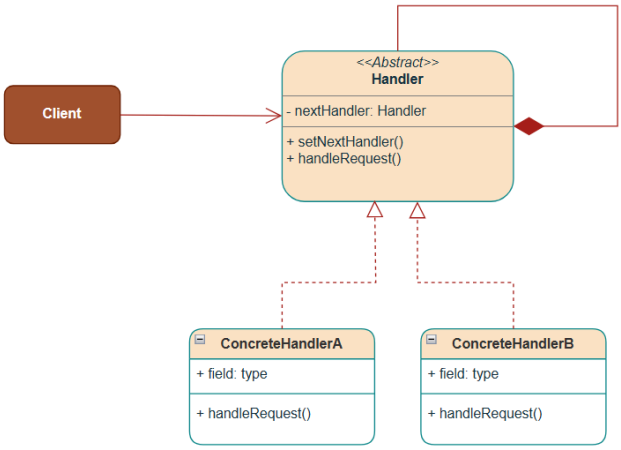
> ***The Chain of Responsibility pattern is a behavioral design pattern that passes requests along a chain of handlers. Each handler decides either to process the request or pass it to the next handler in the chain.***

Real-life Examples

- ATM Vending Machine
- Application Logging System

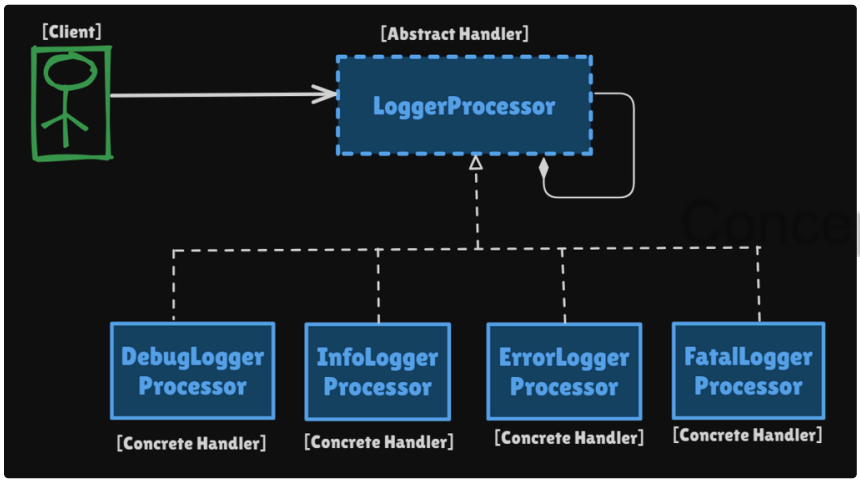


Class Diagram



Structure of CoR Pattern

Let's understand the class diagram using the Logging System example:



1. **Handler Interface (** `LogProcessor` **)**
   - This can be an Abstract Base Class or an Interface.
   - The abstract class defines the common interface and chain mechanism.
   - Declares the `handleRequest()` method.
   - Defines the `setNextHandler()` method.
   - Holds a reference to concrete handlers that implement this interface.
2. **ConcreteHandlers (e.g.,** `DebugLogProcessor` **,** `InfoLogProcessor` **, and so on)**
   - Implement the `Handler` interface.
   - It will handle the request or forward it along to the next handler in the chain.
3. **Client**
   - Composes the chain of handlers using the `setNext()` method dynamically.
   - Initiates the request using the first handler in the chain.

Implementation (Example: Logging System)

```java
// Abstract Logger class - defines the chain structure
public abstract class LogProcessor {

    public static final int DEBUG = 1;
    public static final int INFO = 2;
    public static final int ERROR = 3;
    public static final int FATAL = 4;
    int level;
    LogProcessor nextLoggerProcessor;

    public void setNextLogger(LogProcessor nextLogger) {
        this.nextLoggerProcessor = nextLogger;
    }

    public void logMessage(int level, String message) {
        if (this.level <= level) {
            write(message);
        }

        // Pass to next handler in chain if exists
        if (this.nextLoggerProcessor != null) {
            this.nextLoggerProcessor.logMessage(level, message);
        }
    }

    abstract protected void write(String message);
}
```

```java
// Concrete handler for DEBUG level
public class DebugLogProcessor extends LogProcessor {
    public DebugLogProcessor(int level) {
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("DEBUG: " + message);
    }
}
```



```java
// Concrete handler for INFO level
public class InfoLogProcessor extends LogProcessor {
    public InfoLogProcessor(int level) {
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("INFO: " + message);
    }
}
```

```java
// Concrete handler for ERROR level
public class ErrorLogProcessor extends LogProcessor {
    public ErrorLogProcessor(int level) {
        this.level = level;
    }

    @Override
    protected void write(String message) {
        System.out.println("ERROR: " + message);
    }
}
```

```java
// Concrete handler for FATAL level
public class FatalLogProcessor extends LogProcessor {
    public FatalLogProcessor(int level) {
        this.level = level;
```

```
    }

    @Override
    protected void write(String message) {
        System.out.println("FATAL: " + message);
    }
}
```

```
// Client code
public class LoggerDemo {
    public static void main(String[] args) {
        System.out.println("###### Chain of Responsibility Design Pattern ######");

        // Get the chain of loggers
        LogProcessor logProcessor = getChainOfLoggers();

        System.out.println("Logging messages:");
        System.out.println("===== Logging DEBUG message =====");
        logProcessor.logMessage(LogProcessor.DEBUG, "This is a debug message");
        System.out.println("===== Logging INFO message =====");
        logProcessor.logMessage(LogProcessor.INFO, "This is an info message");
        System.out.println("===== Logging ERROR message =====");
        logProcessor.logMessage(LogProcessor.ERROR, "This is an error message");
        System.out.println("===== Logging FATAL message =====");
        logProcessor.logMessage(LogProcessor.FATAL, "This is a fatal message");
    }

    private static LogProcessor getChainOfLoggers() {
        LogProcessor fatalLogger = new FatalLogProcessor(LogProcessor.FATAL); // 4
        LogProcessor errorLogger = new ErrorLogProcessor(LogProcessor.ERROR); // 3
        LogProcessor infoLogger = new InfoLogProcessor(LogProcessor.INFO); // 2
        LogProcessor debugLogger = new DebugLogProcessor(LogProcessor.DEBUG); // 1

        // Dynamic Chaining: DEBUG -> INFO -> ERROR -> FATAL
        debugLogger.setNextLogger(infoLogger);
        infoLogger.setNextLogger(errorLogger);
        errorLogger.setNextLogger(fatalLogger);
        // fatalLogger.nextLoggerProcessor is null; // Last logger in chain

        return debugLogger;  // Return the first LogProcessor in the chain
    }
}
```

Output

```
###### Chain of Responsibility Design Pattern ######
Logging messages:
===== Logging DEBUG message =====
DEBUG: This is a debug message
===== Logging INFO message =====
DEBUG: This is an info message
INFO: This is an info message
===== Logging ERROR message =====
DEBUG: This is an error message
INFO: This is an error message
ERROR: This is an error message
===== Logging FATAL message =====
DEBUG: This is a fatal message
INFO: This is a fatal message
ERROR: This is a fatal message
FATAL: This is a fatal message

Process finished with exit code 0
```

Command Pattern

Resources

- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 31. Design Undo, Redo feature with Command Pattern | Command Design Pattern| Low Level System Design

Definition

> ***The Command pattern is a behavioral design pattern that encapsulates a request as an object, allowing you to parameterize and queue them, which in turn helps in decoupling the sender and receiver. By storing the object's state, we can also implement undo/redo operations.***

Let's consider an example of Remote controls for devices, like sending commands from an AC Remote to the actual AC (Air Conditioner) device set up in the room. The remote control doesn't need to know how the AC works internally - it just calls `execute()` upon a button press(command). We can introduce parameterization by having different buttons configured with different commands at runtime. The pattern separates the object that invokes the operation(sender) from the one that performs it(receiver), making the system more flexible and maintainable.

The Problem: Traditional Approach

Let's implement the above example in a naive approach.

```java
public class AirConditioner {
    boolean isOn;
    int temperature;

    public void turnOn() {
        isOn = true;
        System.out.println("Air conditioner is on");
    }

    public void turnOff() {
        isOn = false;
        System.out.println("Air conditioner is off");
    }

    public void setTemperature(int temperature) {
        this.temperature = temperature;
        System.out.println("Air conditioner temperature set to " +
temperature);
    }
}
```

```java
public class Bulb {
    boolean isOn;

    public void turnOn() {
        isOn = true;
        System.out.println("Bulb is on");
    }

    public void turnOff() {
        isOn = false;
        System.out.println("Bulb is off");
    }
}
```

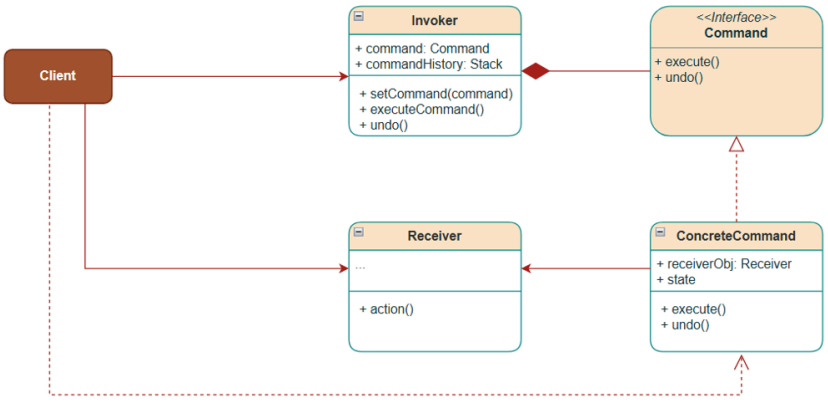
```java
public class Client {
    public static void main(String[] args) {
        System.out.println("Command Pattern: Problem Demo");

        // Device: Air Conditioner Commands
        AirConditioner airConditioner = new AirConditioner();
        airConditioner.turnOn();
        airConditioner.setTemperature(25);
        airConditioner.turnOff();

        // Device: Bulb Commands
        Bulb bulb = new Bulb();
        bulb.turnOn();
        bulb.turnOff();
    }
}
```

The problems with the above implementation are:

1. **Lack of Abstraction:** The remote control is directly dependent on specific device classes. Tomorrow, when we want to scale to a smart controller by adding new devices, we will need to modify the remote control code. This will lead to more redundant code, which is not a good design practice.
2. **Undo/Redo Functionality:** What if we want to add the undo/redo capability? How it will be handled. If we provide the implementation in client code (without command objects storing previous state), implementing undo becomes clumsy, requiring the invoker to track state for all possible operations.
3. **Difficulty in Code Maintenance:** What if in the future, we have to support more commands for more devices example Bulb. Supporting multiple device types leads to complex, monolithic, bloated remote control classes, leading to violation of SOLID Principles and difficulty in Testing.

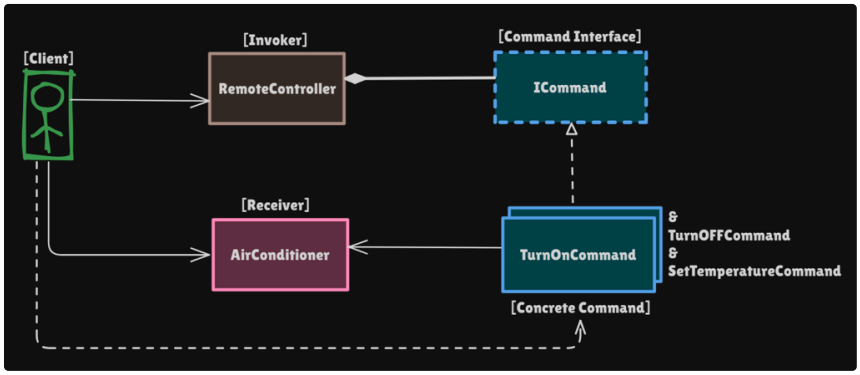
Without this pattern, you end up with tightly coupled, inflexible code that's hard to maintain, test, and extend.

Class Diagram



Structure of the Command Pattern

Understanding the structure of the command design pattern using the AC Remote Control example.



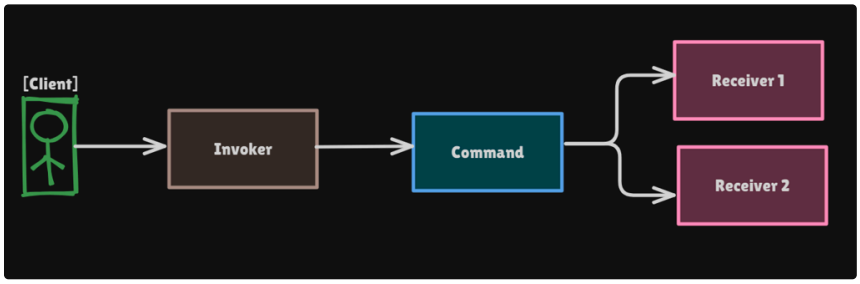
- **Receiver (** `AirConditioner` **)**: The object that performs the actual work. It contains the business logic that will be triggered by a command.
- **Command Interface**: Declares `execute()` and `undo()` methods and introduces a layer of abstraction.
- **Concrete Commands(** `TurnOnCommand` , `TurnOffCommand` , `SetTemperatureCommand` **)**: Implement specific operations like turning on/off, changing temperature. Each receiver operation has a dedicated class.
- **Invoker (** `RemoteController` **)**: Stores a reference to the Command object and executes commands upon receiving a request for an operation. Holds a data structure to track previous states that help implement undo/redo functionality.
- **Client**: Creates and configures commands and associates them with the right receiver, and configures the Invoker to execute the commands.

Implementation (Example: AC Remote with Undo Functionality)

How does the Command design pattern solve the above issue?

It separates the logic of:

- Receiver
- Invoker and
- Command




```
// Receiver - The AC Device that performs actual operations
public class AirConditioner {
    boolean isOn;
    int temperature;

    public void turnOn() {
        isOn = true;
```

```
        System.out.println("Air conditioner is on");
    }

    public void turnOff() {
        isOn = false;
        System.out.println("Air conditioner is off");
    }

    public boolean isOn() {
        return isOn;
    }

    public void setOn(boolean on) {
        isOn = on;
    }

    public int getTemperature() {
        return temperature;
    }

    public void setTemperature(int temperature) {
        this.temperature = temperature;
        System.out.println("Air conditioner temperature set to " +
temperature + "°C");
    }

}
```

```
// Command interface
public interface ICommand {
    void execute();

    void undo();
}
```



```
// Concrete Commands
public class TurnOnCommand implements ICommand {
    private final AirConditioner ac;
    private boolean previousState;

    public TurnOnCommand(AirConditioner ac) {
        this.ac = ac;
    }

    @Override
    public void execute() {
        previousState = ac.isOn();
        ac.turnOn();
    }

    @Override
    public void undo() {
        System.out.print("Undo: Turn On command. ");
        if (!previousState) {
            ac.turnOff();
        }
    }
}
```

```
// Concrete Command
public class TurnOffCommand implements ICommand {
    private final AirConditioner ac;
    private boolean previousState;

    public TurnOffCommand(AirConditioner ac) {
        this.ac = ac;
    }

    @Override
    public void execute() {
```

```
        previousState = ac.isOn();
        ac.turnOff();
    }

    @Override
    public void undo() {
        System.out.print("Undo: Turn Off command. ");
        if (previousState) {
            ac.turnOn();
        }
    }
}
```

```
// Concrete Command
public class SetTemperatureCommand implements ICommand {
    private final AirConditioner ac;
    private final int newTemperature;
    private int previousTemperature;

    public SetTemperatureCommand(AirConditioner ac, int temperature) {
        this.ac = ac;
        this.newTemperature = temperature;
    }

    @Override
    public void execute() {
        previousTemperature = ac.getTemperature();
        ac.setTemperature(newTemperature);
    }

    @Override
    public void undo() {
        System.out.print("Undo: Set Temperature Command. ");
        ac.setTemperature(previousTemperature);
    }
}
```



```
// Invoker - with undo functionality
public class RemoteController {
    ICommand command; // Holds reference to the command
    Stack<ICommand> commandHistory = new Stack<>();

    RemoteController() {
    }

    public void setCommand(ICommand command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
        commandHistory.push(command);
    }

    public void undo() {
        if (!commandHistory.isEmpty()) {
            ICommand lastCommand = commandHistory.pop();
            lastCommand.undo();
        }
    }
}
```

```
// Client - Demonstration
public class Client {
    public static void main(String[] args) {
        System.out.println("##### Command Pattern: Solution Demo
#####");

        // Create Receiver
        AirConditioner airConditioner = new AirConditioner();
```

```
8
9           // Create Invoker
10          RemoteController remoteObj = new RemoteController();
11
12          // Execute Commands
13          remoteObj.setCommand(new TurnOnCommand(airConditioner));
14          remoteObj.pressButton();
15          remoteObj.setCommand(new SetTemperatureCommand(airConditioner,
    25));
16          remoteObj.pressButton();
17          remoteObj.setCommand(new SetTemperatureCommand(airConditioner,
    18));
18          remoteObj.pressButton();
19          remoteObj.setCommand(new TurnOffCommand(airConditioner));
20          remoteObj.pressButton();
21
22          // Undo Command
23          remoteObj.undo(); // Undo: Turn Off command => AC is now on
24
25          // Undo Command
26          remoteObj.undo(); // Undo: Set Temperature Command. AC
    temperature is now 25°C
27
28          // Undo Command
29          remoteObj.undo(); // Undo: Set Temperature Command. AC
    temperature is now 0°C
30
31          // Undo Command
32          remoteObj.undo(); // Undo: Turn On command => AC is now off
33      }
34  }
```

Output

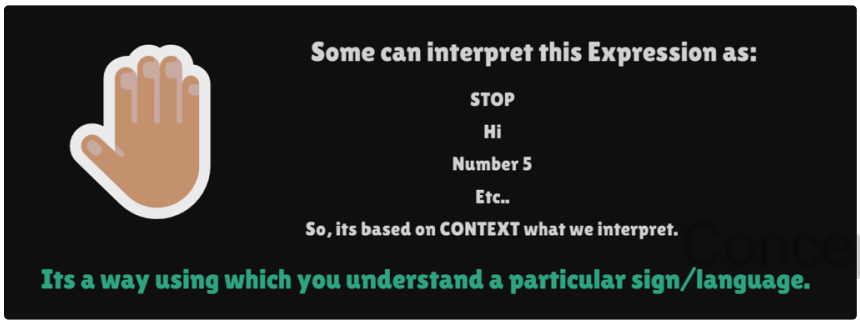
```
##### Command Pattern: Solution Demo #####
Air conditioner is on
Air conditioner temperature set to 25°C
Air conditioner temperature set to 18°C
Air conditioner is off
Undo: Turn Off command. Air conditioner is on
Undo: Set Temperature Command. Air conditioner temperature set to 25°C
Undo: Set Temperature Command. Air conditioner temperature set to 0°C
Undo: Turn On command. Air conditioner is off

Process finished with exit code 0
```

Interpreter Pattern

Resources

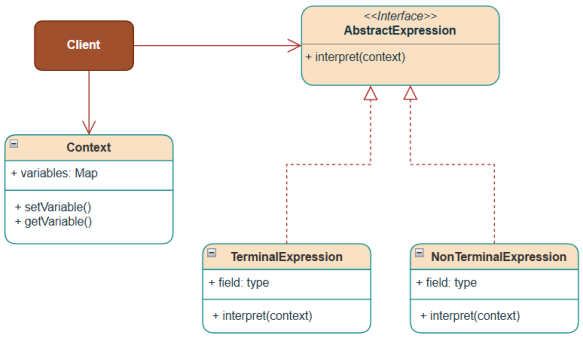
- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 40. Interpreter Design Pattern | LLD System Design | Design pattern explanation in Java

Definition



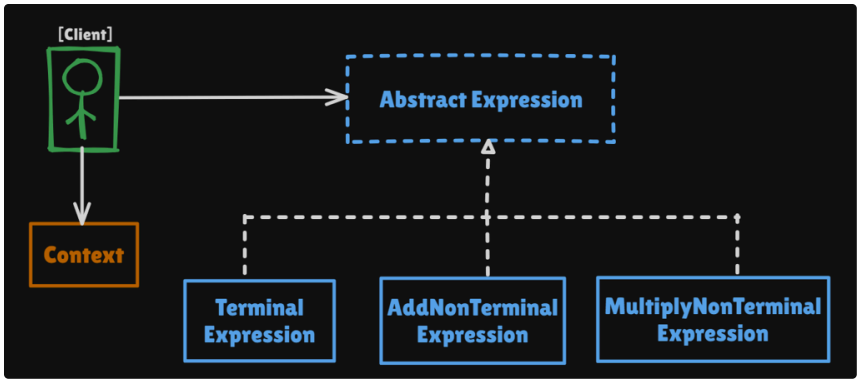
> ***The Interpreter design pattern is a behavioral pattern that defines how to interpret and evaluate expressions.***

Class Diagram



Structure of Interpreter Pattern

Structure of a simple mathematical expression interpreter that can handle addition and multiplication of numbers:



- **Context**: Contains information that's global to the interpreter (like variable values).
- **Abstract Expression**: Defines the interpret operation that all concrete expressions must implement.
- **Terminal Expression**: Implements interpretation for variables in the expressions (leaf nodes).
- **Non-terminal Expression(** `AddNonTerminalExpression`, `MultiplyNonTerminalExpression` **)**: Implements interpretation for non-terminal expressions (composite expressions that contain other expressions).
- **Client**: Builds the expressions and invokes the interpret operation.

Implementation

Let's create a simple mathematical expression interpreter that can handle addition and multiplication of numbers:

```java
// Context class
class Context {
    private final Map<String, Integer> variables = new HashMap<>();

    public void setVariable(String name, int value) {
        variables.put(name, value);
    }

    public int getVariable(String name) {
        return variables.getOrDefault(name, 0);
    }

    @Override
    public String toString() {
        return variables.toString();
    }
}
```



```java
// Abstract Expression interface
public interface AbstractExpression {
    int interpret(Context context);
}
```

```java
// Terminal Expression - represents a variable
public class NumberTerminalExpression implements AbstractExpression {
    String stringValue;

    NumberTerminalExpression(String stringVal) {
        this.stringValue = stringVal;
    }

    @Override
    public int interpret(Context context) {
        return context.getVariable(stringValue);
    }
}
```

```java
// Non-terminal Expression - represents addition
public class AddNonTerminalExpression implements AbstractExpression {
    private final AbstractExpression rightExpression;
    private final AbstractExpression leftExpression;

    public AddNonTerminalExpression(AbstractExpression left,
AbstractExpression right) {
        this.leftExpression = left;
        this.rightExpression = right;
    }

    @Override
    public int interpret(Context context) {
        return leftExpression.interpret(context) +
rightExpression.interpret(context);
    }
}
```

```java
// Non-terminal Expression - represents multiplication
public class MultiplyNonTerminalExpression implements
AbstractExpression {
    private final AbstractExpression leftExpression;
    private final AbstractExpression rightExpression;

    public MultiplyNonTerminalExpression(AbstractExpression left,
AbstractExpression right) {
        this.leftExpression = left;
        this.rightExpression = right;
    }

    @Override
    public int interpret(Context context) {
        return leftExpression.interpret(context) *
rightExpression.interpret(context);
    }
}
```



```java
// Non Terminal Expression - represents an expression with a binary
operator + or *
public class BinaryNonTerminalExpression implements AbstractExpression
{
    AbstractExpression leftExpression;
    AbstractExpression rightExpression;
    char operator;

    public BinaryNonTerminalExpression(AbstractExpression
leftExpression, AbstractExpression rightExpression, char operator) {
        this.leftExpression = leftExpression;
        this.rightExpression = rightExpression;
        this.operator = operator;
    }

    @Override
    public int interpret(Context context) {
        return switch (operator) {
            case '+' -> leftExpression.interpret(context) +
rightExpression.interpret(context);
            case '*' -> leftExpression.interpret(context) *
rightExpression.interpret(context);
            default -> 0;
        };
    }
}
```

```java
// Client code
public class Client {
    public static void main(String[] args) {
        System.out.println("##### Interpreter Design Pattern #####");

        // Context
```

```
        Context context = new Context();
        context.setVariable("a", 12);
        context.setVariable("b", 5);
        context.setVariable("c", 3);
        context.setVariable("d", 9);
        System.out.println("Context: " + context);

        // Expression: a + b
        AbstractExpression expression1 = new AddNonTerminalExpression(
                new NumberTerminalExpression("a"),
                new NumberTerminalExpression("b"));
        System.out.println("Expression: (a+b) = " +
expression1.interpret(context)); // Output: 17

        // Expression: a * b
        AbstractExpression expression2 = new
MultiplyNonTerminalExpression(
                new NumberTerminalExpression("a"),
                new NumberTerminalExpression("b")
        );
        System.out.println("Expression: (a*b) = " +
expression2.interpret(context)); // Output: 60

        // Complex Expression: (a + b) * c
        AbstractExpression expression3 = new
MultiplyNonTerminalExpression(
                new AddNonTerminalExpression(
                        new NumberTerminalExpression("a"),
                        new NumberTerminalExpression("b")
                ),
                new NumberTerminalExpression("c")
        );
        System.out.println("Expression: ((a+b)*c) = " +
expression3.interpret(context)); // Output: 51

        // Expression: ((a*b) + (c*d))
        AbstractExpression expression4 = new
BinaryNonTerminalExpression(
                new BinaryNonTerminalExpression(
                        new NumberTerminalExpression("a"), new
NumberTerminalExpression("b"), '*'),
                new BinaryNonTerminalExpression(
                        new NumberTerminalExpression("c"), new
NumberTerminalExpression("d"), '*'),
                '+');
        System.out.println("Expression: ((a*b) + (c*d)) = " +
expression4.interpret(context));

    }
}
```


Output

```
##### Interpreter Design Pattern #####
Context: {a=12, b=5, c=3, d=9}
Expression: (a+b) = 17
Expression: (a*b) = 60
Expression: ((a+b)*c) = 51
Expression: ((a*b) + (c*d)) = 87

Process finished with exit code 0
```

Iterator Pattern

Resources

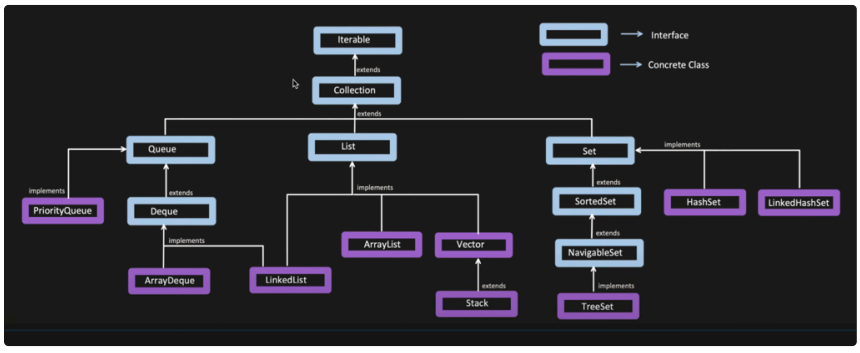
- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 33. Iterator Design Pattern Explained with Example | Low Level Design

Definition

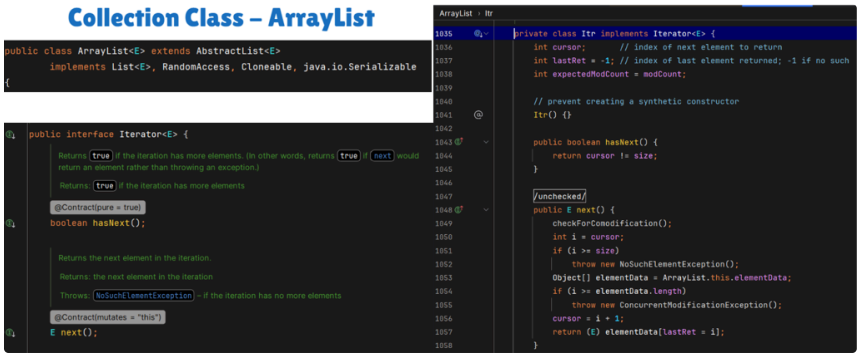
> **The Iterator design pattern is one of the behavioral design patterns that provides a way to access elements of a Collection sequentially without exposing the underlying representation of the collection.**

Real-world Usage

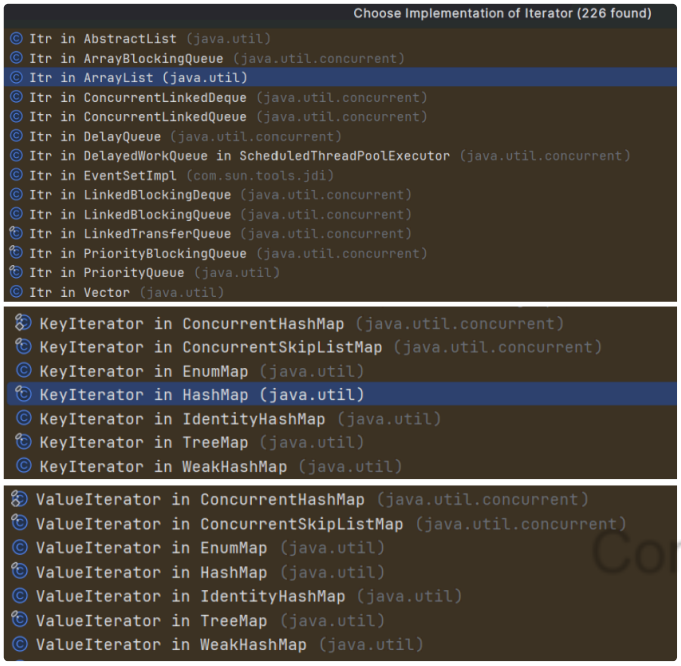
In the Java Collections Framework, the collection interface includes methods, such as `iterator()`, that allow a client to obtain an Iterator object from any collection that implements it. All collection classes have iterator implementation, e.g., (`ArrayList.iterator()`, `HashSet.iterator()`, `TreeSet.descendingIterator()`, etc).



Java Collections Framework



ArrayList implementation of Iterator



Iterator Interface Implementations

Code Example

```java
// Java Collections Usage Example
public class LinkedHashSetExample {
    public static void main(String[] args) {
        Set<Integer> intSet = new LinkedHashSet<>();
        intSet.add(2);
        intSet.add(77);
        intSet.add(82);
        intSet.add(63);
        intSet.add(5);

        // Common to all Collection Classes
        Iterator<Integer> iterable = intSet.iterator();
        while (iterable.hasNext()) {
            int value = iterable.next();
            System.out.println(value);
        }
    }
}
```

The Problem: Data exposure to the Client

```java
public class Book {
    private final String title;
    private final String author;
    private final String isbn;
```

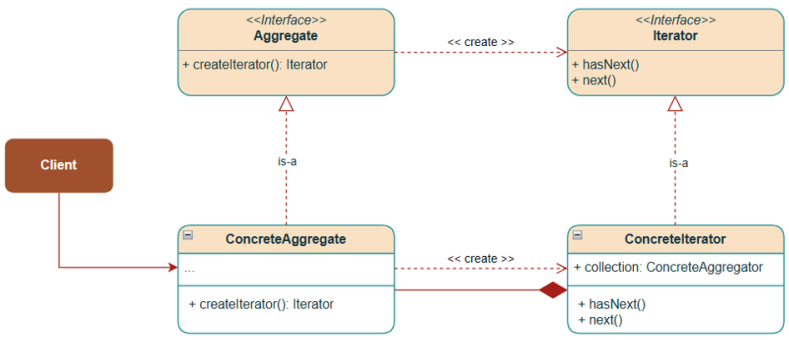
```
5
6       public Book(String tittle, String author, String isbn) {
7           this.title = tittle;
8           this.author = author;
9           this.isbn = isbn;
10      }
11
12      public static List<Book> getBooks() {
13          List<Book> books = List.of(
14                  new Book("To Kill a Mockingbird", "Harper Lee", "978-
    0-74-7356-5"),
15                  new Book("The Great Gatsby", "F. Scott Fitzgerald",
    "778-0-24-7156-5"),
16                  new Book("The Catcher in the Rye", "J.D. Salinger",
    "333-0-28-7446-8"),
17                  new Book("The Hobbit", "J.R.R. Tolkien", "783-0-14-
    1951-8"),
18                  new Book("Rich Dad Poor Dad", "Robert Kiyosaki", "183-
    0-12-1491-8"),
19                  new Book("Pride and Prejudice", "Jane Austen", "289-0-
    12-1678-8")
20          );
21          return books;
22      }
23
24      @Override
25      public String toString() {
26          return "Book [" + "Tittle='" + title + ", Author=" + author +
    ", age=" + "ISBN=" + isbn + "]";
27      }
28  }
```

```
1  public class Client {
2      public static void main(String[] args) {
3          System.out.println("\n###### Problem without Iterator Pattern
   Demo ######");
4          // Client has access to the entire book list in the library
5          List<Book> bookList = Book.getBooks();
6          for (Book book : bookList) {
7              System.out.println(book);
8          }
9      }
10 }
```


Problems with the above code are that:

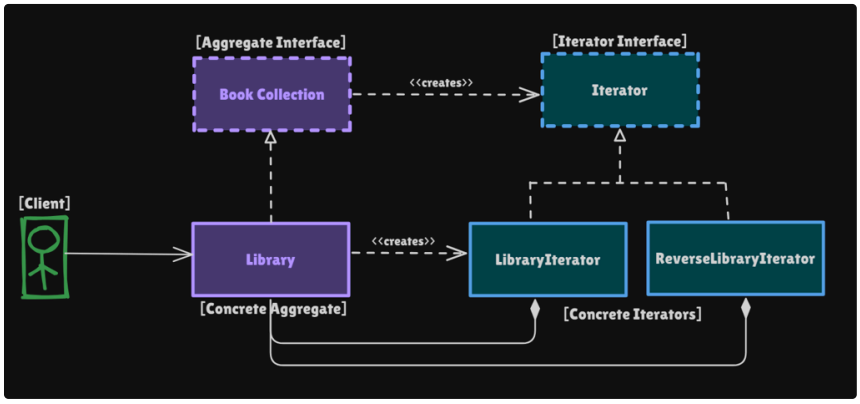
- No data encapsulation allows the client can modify data and access hidden information.
- Tight coupling of traversal logic to data structure and client.
- Violates SOLID principles if we try to implement new traversal logic.

Class Diagram



Structure of the Iterator Design Pattern

Let's understand the Structure of the Iterator Design Pattern using a Library example:



- **Iterator Interface(** `Iterator<T>` **)**: Defines the contract for traversing a collection with `hasNext()` and `next()` methods.
- **Concrete Iterator**: Implements the traversal logic for a specific collection type. It tracks the current position while traversing the collection.
  - `LibraryIterator` → traverses books from first to last.
  - `DescendingLibraryIterator` → traverses books from last to first by starting at `numberOfBooks - 1` and decrementing the position.
- **Aggregate Interface(** `BookCollection` **)**: Declares a method to create iterators (`createIterator()` and `createReverseIterator()`).
- **Concrete Aggregate(** `Library` **)**: The actual collection of books (`Library` class) that implements the aggregate interface(creates and returns **Concrete Iterators** `LibraryIterator` **and** `DescendingLibraryIterator`).

Implementation(Example: Library)

```java
// Book class representing individual books in a library
public class Book {
    private final String title;
    private final String author;
    private final String isbn;
    private int price;

    public Book(String title, String author, String isbn) {
        this.title = title;
        this.author = author;
        this.isbn = isbn;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }

    public String getIsbn() {
        return isbn;
    }
```

```
25
26      public int getPrice() {
27          return price;
28      }
29
30      @Override
31      public String toString() {
32          return "Book [Title=" + title + ", Author=" + author + ",
 ISBN=" + isbn + "]";
33      }
34  }
```

```
1  // Aggregate interface
2  public interface BookCollection {
3      Iterator<Book> createIterator();
4
5      Iterator<Book> createReverseIterator();
6  }
```

```
1  // Iterator interface
2  public interface Iterator<T> {
3      boolean hasNext();
4
5      T next();
6  }
```

```
1  // Concrete Aggregate
2  public class Library implements BookCollection {
3
4      private final List<Book> books;
5
6      public Library(List<Book> books) {
7          this.books = books;
8      }
9
10     @Override
11     public Iterator<Book> createIterator() {
12         return new LibraryIterator(books);
13     }
14
15     @Override
16     public Iterator<Book> createReverseIterator() {
17         return new ReverseLibraryIterator(books);
18     }
19 }
```



```
1  // Concrete Iterator - for Library
2  public class LibraryIterator implements Iterator<Book> {
3      private final List<Book> books;
4      private int position = 0;
5
6      public LibraryIterator(List<Book> books) {
7          this.books = books;
8      }
9
10     @Override
11     public boolean hasNext() {
12         return position < books.size();
13     }
14
15     @Override
16     public Book next() {
17         return books.get(position++);
18     }
19 }
```

```
1  // Concrete Iterator - for Library
2  public class ReverseLibraryIterator implements Iterator<Book> {
3      private final List<Book> books;
4      private int position;
```

```
5
6       public ReverseLibraryIterator(List<Book> books) {
7           this.books = books;
8           this.position = books.size() - 1;
9       }
10
11      @Override
12      public boolean hasNext() {
13          return position >= 0 && books.get(position) != null;
14      }
15
16      @Override
17      public Book next() {
18          if (!hasNext()) {
19              return null;
20          }
21          return books.get(position--); // Return current book and move
    backward
22      }
23  }
```

```
1   // Client
2   public class LibraryIteratorDemo {
3
4       public static void main(String[] args) {
5           System.out.println("\n###### Iterator Design Pattern ######");
6
7           // Create Library
8           Library library = getLibrary();
9
10          // Forward iteration
11          Iterator<Book> iterator = library.createIterator();
12          System.out.println("\n==> Forward iteration:");
13          displayLibrary(iterator);
14
15          // Reverse iteration
16          Iterator<Book> reverseIterator =
    library.createReverseIterator();
17          System.out.println("\n==> Reverse iteration:");
18          displayLibrary(reverseIterator);
19      }
20
21      private static Library getLibrary() {
22          List<Book> books = List.of(
23                  new Book("To Kill a Mockingbird", "Harper Lee", "978-
    0-74-7356-5"),
24                  new Book("The Great Gatsby", "F. Scott Fitzgerald",
    "778-0-24-7156-5"),
25                  new Book("The Catcher in the Rye", "J.D. Salinger",
    "333-0-28-7446-8"),
26                  new Book("The Hobbit", "J.R.R. Tolkien", "783-0-14-
    1951-8"),
27                  new Book("Rich Dad Poor Dad", "Robert Kiyosaki", "183-
    0-12-1491-8"),
28                  new Book("Pride and Prejudice", "Jane Austen", "289-0-
    12-1678-8")
29          );
30          Library library = new Library(books);
31          return library;
32      }
33
34      private static void displayLibrary(Iterator<Book> iterator) {
35          while (iterator.hasNext()) {
36              Book book = iterator.next();
37              System.out.println(book);
38          }
39      }
40  }
```

Output

```
###### Iterator Design Pattern ######

==> Forward iteration:
Book [Title=To Kill a Mockingbird, Author=Harper Lee, ISBN=978-0-74-7356-5]
Book [Title=The Great Gatsby, Author=F. Scott Fitzgerald, ISBN=778-0-24-7156-5]
Book [Title=The Catcher in the Rye, Author=J.D. Salinger, ISBN=333-0-28-7446-8]
Book [Title=The Hobbit, Author=J.R.R. Tolkien, ISBN=783-0-14-1951-8]
Book [Title=Rich Dad Poor Dad, Author=Robert Kiyosaki, ISBN=183-0-12-1491-8]
Book [Title=Pride and Prejudice, Author=Jane Austen, ISBN=289-0-12-1678-8]

==> Reverse iteration:
Book [Title=Pride and Prejudice, Author=Jane Austen, ISBN=289-0-12-1678-8]
Book [Title=Rich Dad Poor Dad, Author=Robert Kiyosaki, ISBN=183-0-12-1491-8]
Book [Title=The Hobbit, Author=J.R.R. Tolkien, ISBN=783-0-14-1951-8]
Book [Title=The Catcher in the Rye, Author=J.D. Salinger, ISBN=333-0-28-7446-8]
Book [Title=The Great Gatsby, Author=F. Scott Fitzgerald, ISBN=778-0-24-7156-5]
Book [Title=To Kill a Mockingbird, Author=Harper Lee, ISBN=978-0-74-7356-5]

Process finished with exit code 0
```

Mediator Pattern

Resources

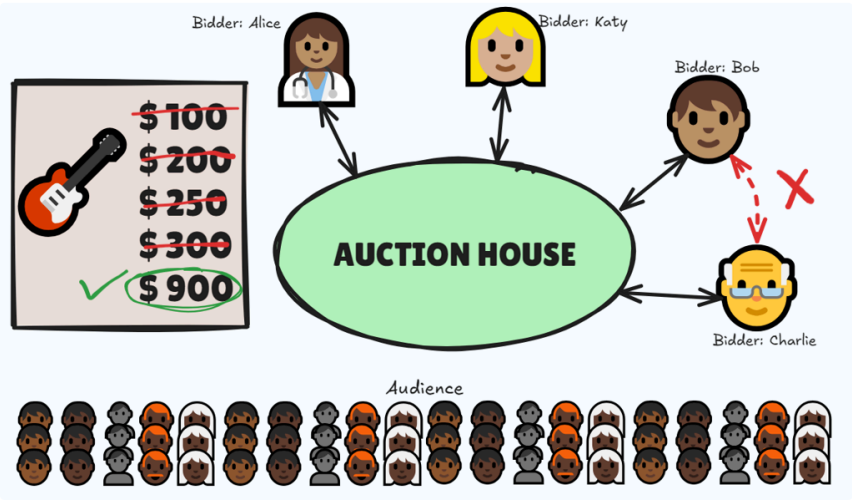
- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 34. Design Online Auction System with Mediator Design Pattern | Low Level System Design

Definition

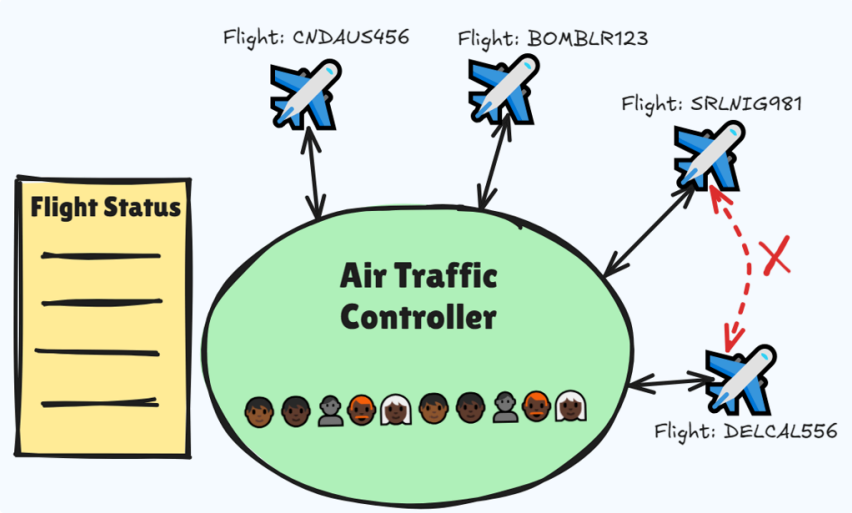
> **The Mediator design pattern is a beahvioral pattern that defines a mediator object that encapsulates the behavior of how a set of objects(components) interact. It promotes loose coupling by not allowing these objects from referring to each other explicitly but allows them to interact through mediator object upon respective state changes/updates.**

Real life Examples

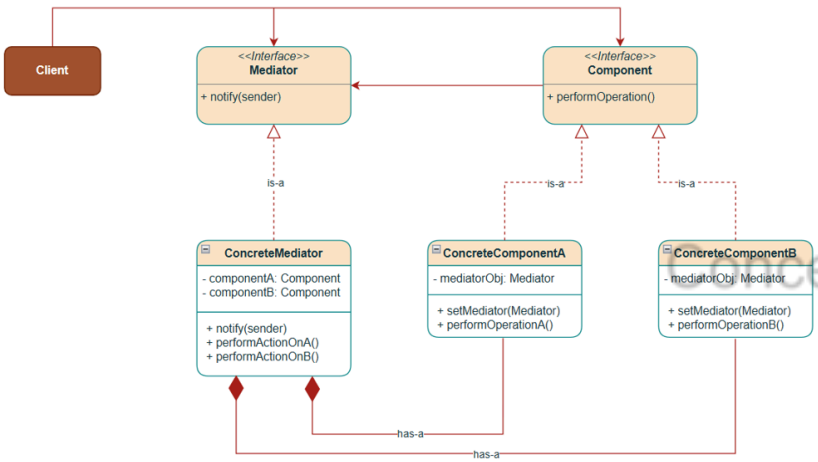
Online Auction System



Airline Management System

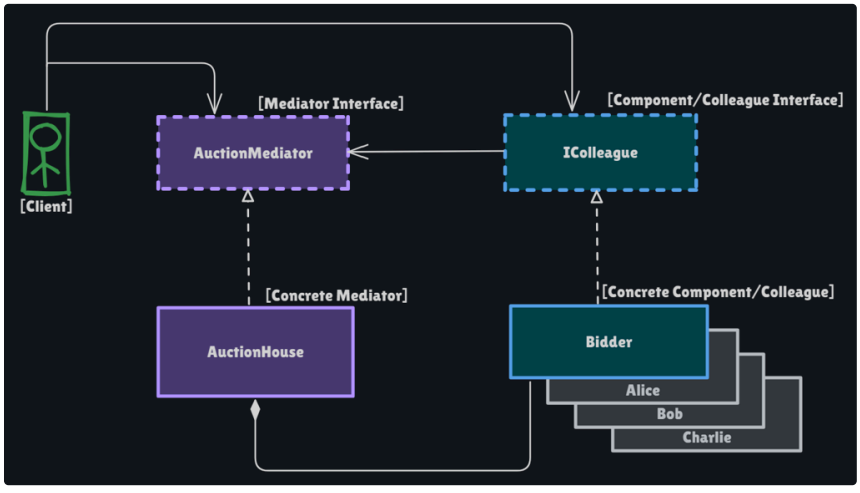


Class Diagram



Structure of Mediator Pattern

Let's understand the Online Auction System using Online Auction System Example



1. **Mediator Interface (`AuctionMediator`):** Defines the contract for communication between bidders. Methods include:
   - `registerBidder()` - Add bidders to the auction
   - `placeBid()` - Process bids from bidders
   - `closeAuction()` - End the auction and announce winner
2. **Concrete Mediator (`AuctionHouse`):** Implements Mediator Interface behaviors and maintains references to all bidders.
   - Tracks the highest bid and bidder and encapsulates the core bidding logic of interaction.
   - Validates bids and notifies all participants upon appropriate state changes in bidders(components).
3. **Colleague/Component Interface (`IColleague`):** Abstract class or Interface representing the components (colleagues/bidders/auction participants) that vary independently.
   - Holds reference to the mediator (not to other bidders).
   - Declares contracts for performing operations (placing bids and receiving notifications).
4. **Concrete Colleague/Component** (`Bidder`)
   - Defines methods for performing operations (actual bidder implementation).
   - This class communicates only with the mediator.
   - Receives notifications about updates/state changes.

Implementation(Example: Online Auction System)


```java
// Mediator Interface
public interface AuctionMediator {
    void registerBidder(IColleague bidder);

    void placeBid(IColleague bidder, double bidAmount);

    void closeAuction();
}
```

```java
// Concrete Mediator
public class AuctionHouse implements AuctionMediator {
    private List<IColleague> bidders;
    private String itemName;
    private double currentHighestBid;
    private IColleague currentHighestBidder;

    public AuctionHouse(String itemName, double startingPrice) {
        this.itemName = itemName;
        this.currentHighestBid = startingPrice;
        this.bidders = new ArrayList<>();
        System.out.println("[+] Auction House created for item: " +
itemName +
                           " with initial bid of $" + startingPrice);
    }

    @Override
    public void registerBidder(IColleague bidder) {
        bidders.add(bidder);
        System.out.println("[+] " + bidder.getName() + " has joined
the auction
                           for " + itemName);
    }

    @Override
    public void placeBid(IColleague bidder, double bidAmount) {
        // Check if the bid is valid
```

```
        if (bidAmount <= currentHighestBid) {
            System.out.println(bidder.getName() + " bid of $" +
bidAmount +
                               " is too low. Current highest bid is $"
                               + currentHighestBid);
            // Colleagues are not notified about the bid
            return;
        }

        // Update the highest bid
        currentHighestBid = bidAmount;
        currentHighestBidder = bidder;
        System.out.println("\n===> [New Bid Accepted]" + " Info:
{Bidder: "
                           + bidder.getName() + ", Bid Amount: " +
bidAmount + "}");
        for (IColleague colleague : bidders) {
            if (!colleague.getName().equals(bidder.getName())) {
                // Notify other bidders about the new bid
                colleague.receiveBidNotification(bidAmount);
            }
        }
    }

    @Override
    public void closeAuction() {
        if (currentHighestBidder != null) {
            System.out.println("\n===> [AUCTION UPDATE]");
            System.out.println("[+] Auction closed! Winner is "
                               + currentHighestBidder.getName() +
                    " with a bid of $" + currentHighestBid + " for " +
itemName);
        } else {
            System.out.println("Auction closed with no bids.");
        }
    }

}
```



```
// Colleague Interface(aka Component Interface)
public interface IColleague {
    void placeBid(double amount);

    void receiveBidNotification(double bidAmount);

    String getName();
}
```

```
// Concrete Colleague/Component
public class Bidder implements IColleague {
    protected String name;
    protected AuctionMediator mediator;

    public Bidder(String name, AuctionMediator mediator) {
        this.name = name;
        this.mediator = mediator;
        mediator.registerBidder(this);
    }

    @Override
    public void placeBid(double amount) {
        System.out.println("\n===> [Placing Bid] " + name
                           + " is attempting to bid $" + amount);
        mediator.placeBid(this, amount);
    }

    @Override
    public void receiveBidNotification(double bidAmount) {
        System.out.println("[+] Bidder " + name +
                           " has received a new bid notification of: "
+ bidAmount);
```

```
    }

    @Override
    public String getName() {
        return name;
    }
}
```

```
// Client
public class AuctionDemo {
    public static void main(String[] args) {
        System.out.println("\n###### Mediator Design Pattern ######");
        System.out.println("\n===> Welcome to the Auction House!\n");

        // Create a Mediator
        AuctionMediator auctionHouse = new AuctionHouse("Vintage Guitar", 100.0);

        // Create Colleagues/Components
        IColleague alice = new Bidder("Alice", auctionHouse);
        IColleague bob = new Bidder("Bob", auctionHouse);
        IColleague charlie = new Bidder("Charlie", auctionHouse);

        // Register Colleagues/Components with Mediator - AuctionHouse Constructor

        // Use Colleagues/Components
        alice.placeBid(150.0);
        bob.placeBid(250.0);
        charlie.placeBid(300.0);
        alice.placeBid(300.0); // Will not be accepted
        bob.placeBid(900.0); // Winner

        // Admin closes the auction
        auctionHouse.closeAuction();

    }

}
```


Output

```
###### Mediator Design Pattern ######

===> Welcome to the Auction House!

[+] Auction House created for item: Vintage Guitar with initial bid of $100.0
[+] Alice has joined the auction for Vintage Guitar
[+] Bob has joined the auction for Vintage Guitar
[+] Charlie has joined the auction for Vintage Guitar
```

```
===> [Placing Bid] Alice is attempting to bid $150.0

===> [New Bid Accepted] Info: {Bidder: Alice, Bid Amount: 150.0}
[+] Bidder Bob has received a new bid notification of: 150.0
[+] Bidder Charlie has received a new bid notification of: 150.0

===> [Placing Bid] Bob is attempting to bid $250.0

===> [New Bid Accepted] Info: {Bidder: Bob, Bid Amount: 250.0}
[+] Bidder Alice has received a new bid notification of: 250.0
[+] Bidder Charlie has received a new bid notification of: 250.0

===> [Placing Bid] Charlie is attempting to bid $300.0

===> [New Bid Accepted] Info: {Bidder: Charlie, Bid Amount: 300.0}
[+] Bidder Alice has received a new bid notification of: 300.0
[+] Bidder Bob has received a new bid notification of: 300.0

===> [Placing Bid] Alice is attempting to bid $300.0
Alice bid of $300.0 is too low. Current highest bid is $300.0

===> [Placing Bid] Bob is attempting to bid $900.0

===> [New Bid Accepted] Info: {Bidder: Bob, Bid Amount: 900.0}
[+] Bidder Alice has received a new bid notification of: 900.0
[+] Bidder Charlie has received a new bid notification of: 900.0

===> [AUCTION UPDATE]
[+] Auction closed! Winner is Bob with a bid of $900.0 for Vintage Guitar
```

Memento Pattern

Resources

- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 38. Memento Design Pattern explanation | LLD System Design | Design pattern explanation in Java

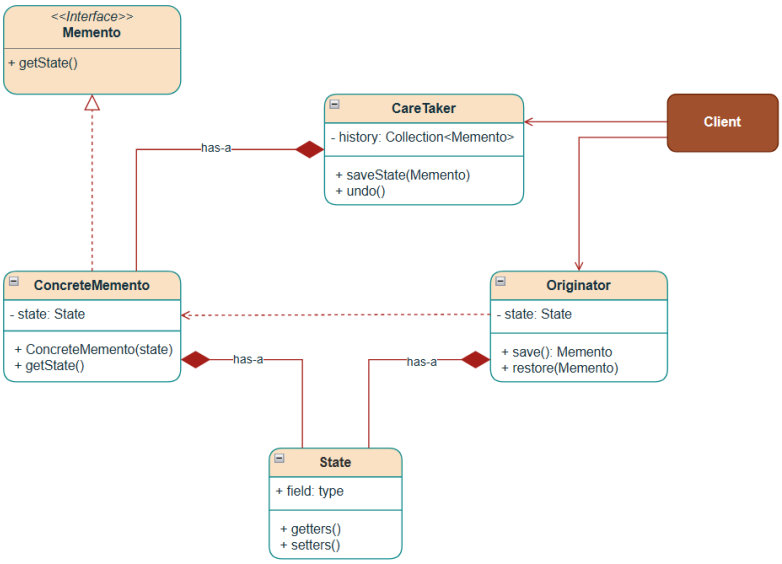
Definition

> ***The Memento pattern is a behavioral design pattern that allows you to capture and restore an object's internal state without violating encapsulation. It's widely used for implementing undo/redo functionality and checkpoints or versioning(systems that require restoring previous states when requested) in your application.***

Memento pattern is made up 3 types of classes, each responsible for particluar action:

1. **Originator**
   - It represents the object, for which state need to be saved and restored.
   - Expose methods to save and restore its state using Memento object.
2. **Memento**
   - It represents an Object which holds the state of the Originator.
3. **Caretaker**
   - Manages the list of States (i.e. list of Memento Objects).

Class Diagram



Structure of Memento Pattern

Understanding the structure of Memento Pattern using an example of IDE application's configuration that can be saved and restored:

1. **Memento (`ConfigurationMemento`)**:
   - Immutable object that stores the configuration state.
   - Only the Originator can access its internal data.
2. **Originator (`ApplicationConfiguration`)**:
   - Contains the actual configuration that changes over time.
   - `save()` method → creates a memento capturing current state
   - `restore()` method → brings back a previous state from a memento.
3. **Caretaker (`ConfigurationManager`)**:
   - Manages a history of mementos for undo functionality.
   - Doesn't know about the internal structure of mementos.
   - Provides clean interface to save and restore states.

Implementation

Here is example of an IDE application's configuration that can be saved and restored:

```
1  // Originator class - creates and restores from mementos
2  public class ApplicationConfiguration {
3      // State
4      private String theme;
5      private int fontSize;
6      private boolean notificationsEnabled;
7      private String language;
8
9      public ApplicationConfiguration(String theme, int fontSize,
10                                     boolean notificationsEnabled,
   String language) {
11         this.theme = theme;
12         this.fontSize = fontSize;
13         this.notificationsEnabled = notificationsEnabled;
```

```
        this.language = language;
    }

    // Create a memento with current state
    public ConfigurationMemento save() {
        System.out.println("[+] Saving configuration state...");
        return new ConfigurationMemento(theme, fontSize,
notificationsEnabled, language);
    }

    // Restore state from memento
    public void restore(ConfigurationMemento memento) {
        this.theme = memento.getTheme();
        this.fontSize = memento.getFontSize();
        this.notificationsEnabled = memento.isNotificationsEnabled();
        this.language = memento.getLanguage();
        System.out.println("[+] Restored Previous Configuration
State");
    }

    // Setters to modify state
    public void setTheme(String theme) {
        this.theme = theme;
    }

    public void setFontSize(int fontSize) {
        this.fontSize = fontSize;
    }

    public void setNotificationsEnabled(boolean enabled) {
        this.notificationsEnabled = enabled;
    }

    public void setLanguage(String language) {
        this.language = language;
    }

    @Override
    public String toString() {
        return String.format("Configuration[Theme=%s, Font Size=%d,
Notifications=%b, Language=%s]",
                theme, fontSize, notificationsEnabled, language);
    }
}
```



```
// Caretaker class - manages mementos
public class ConfigurationManager {
    private final Stack<ConfigurationMemento> history = new Stack<>();

    public void saveState(ApplicationConfiguration appConfig) {
        ConfigurationMemento configurationMemento = appConfig.save();
// creates a memento with current state
        history.push(configurationMemento); // stores the memento in
the history
        System.out.println("[+] State saved. History size: " +
history.size());
        System.out.println(history.size() == 1 ? "[+] Default State: "
+ configurationMemento : "[+] Current State: " +
configurationMemento);
    }

    public void undo(ApplicationConfiguration appConfig) {
        if (history.size() > 1) {
            history.pop(); // removes and returns the last saved state
            ConfigurationMemento mementoToBeRestored = history.peek();
// returns the previous state to be restored
            appConfig.restore(mementoToBeRestored); // restores the
application configuration to the previous saved state
            System.out.println("[+] Undo performed. History size: " +
history.size());
```

```
            System.out.println(history.size() == 1 ? "[+] Default State: " + mementoToBeRestored : "[+] Current State: " + mementoToBeRestored);
        } else {
            System.out.println("[+] No more states to undo!");
            System.out.println("[+] Default State: " + history.peek());
        }
    }
}
```

```
// Memento class - stores the state
public class ConfigurationMemento {
    private final String theme;
    private final int fontSize;
    private final boolean notificationsEnabled;
    private final String language;

    public ConfigurationMemento(String theme, int fontSize,
                                boolean notificationsEnabled, String language) {
        this.theme = theme;
        this.fontSize = fontSize;
        this.notificationsEnabled = notificationsEnabled;
        this.language = language;
    }

    // Getters for restoration
    String getTheme() {
        return theme;
    }

    int getFontSize() {
        return fontSize;
    }

    boolean isNotificationsEnabled() {
        return notificationsEnabled;
    }

    String getLanguage() {
        return language;
    }

    @Override
    public String toString() {
        return String.format("ConfigurationMemento[Theme=%s, Font Size=%d, Notifications=%b, Language=%s]",
                theme, fontSize, notificationsEnabled, language);
    }
}
```



```
// Demo Usage
public class MementoDemo {
    public static void main(String[] args) {
        System.out.println("\n###### Memento Design Pattern ######");

        // Create Originator Object
        ApplicationConfiguration appConfig = new ApplicationConfiguration(
                "Light", 12, true, "English"
        );

        // Create Caretaker Object
        ConfigurationManager configurationManager = new ConfigurationManager();

        // Default State
        System.out.println("\n===> State 1: ");
        configurationManager.saveState(appConfig); // Default State

```

```
        // State 2
        appConfig.setTheme("Dark");
        appConfig.setFontSize(14);
        System.out.println("\n===> State 2: ");
        configurationManager.saveState(appConfig); // Creates a
memento and stores it in history

        // State 3

        appConfig.setTheme("Midnight Blue");
        appConfig.setFontSize(16);
        appConfig.setLanguage("Spanish");
        System.out.println("\n===> State 3: ");
        configurationManager.saveState(appConfig); // Creates a
memento and stores it in history

        // Undo 1
        System.out.println("\n===> Undo 1 ");
        configurationManager.undo(appConfig); // Restores the
application configuration to the previous saved state

        // Undo 2
        System.out.println("\n===> Undo 2: ");
        configurationManager.undo(appConfig); // Restores the
application configuration to the previous saved state

        // Undo 3: Try to undo when no history
        System.out.println("\n===> Undo 3: ");
        configurationManager.undo(appConfig); // Default State
    }
}
```

Output


```
###### Memento Design Pattern ######

===> State 1:
[+] Saving configuration state...
[+] State saved. History size: 1
[+] Default State: ConfigurationMemento[Theme=Light, Font Size=12, Notifications=true, Language=English]

===> State 2:
[+] Saving configuration state...
[+] State saved. History size: 2
[+] Current State: ConfigurationMemento[Theme=Dark, Font Size=14, Notifications=true, Language=English]

===> State 3:
[+] Saving configuration state...
[+] State saved. History size: 3
[+] Current State: ConfigurationMemento[Theme=Midnight Blue, Font Size=16, Notifications=true, Language=Spanish]
```

```
===> Undo 1
[+] Restored Previous Configuration State
[+] Undo performed. History size: 2
[+] Current State: ConfigurationMemento[Theme=Dark, Font Size=14, Notifications=true, Language=English]

===> Undo 2:
[+] Restored Previous Configuration State
[+] Undo performed. History size: 1
[+] Default State: ConfigurationMemento[Theme=Light, Font Size=12, Notifications=true, Language=English]

===> Undo 3:
[+] No more states to undo!
[+] Default State: ConfigurationMemento[Theme=Light, Font Size=12, Notifications=true, Language=English]

Process finished with exit code 0
```

NULL Object Pattern

Resources

- Low Level Design from Basics to Advanced (Some Initial Videos are in Hindi, rest in English)
- 15. LLD of NULL Object Pattern (Hindi) | Design Null Object Pattern | Design Patterns

Understanding the need for the NULL Object Pattern

1. **Problem:** Code without NULL checks leads to unexpected behaviours due to NullPointerExceptions, resulting in a poor user experience. This is also a bad programming practice.

```
private static void printVehicleDetails(Vehicle vehicle){

        System.out.println("Seating Capacity: " + vehicle.getSeatingCapacity());
        System.out.println("Fuel Tank Capacity: " + vehicle.getTankCapacity());
}
```


2. **Solution:** A way to solve this problem is to add NULL checks wherever necessary.

```
private static void printVehicleDetails(Vehicle vehicle) {

    if (vehicle != null) {

        System.out.println("Seating Capacity: " + vehicle.getSeatingCapacity());
        System.out.println("Fuel Tank Capacity: " + vehicle.getTankCapacity());
    }
}
```

Code: Return NULL and NULL Checks Included

```
public abstract class Vehicle {

    public abstract void start();

    public abstract void stop();

}
```

```
public class Car extends Vehicle {
  private String model;
    private String color;
    private int seatingCapacity;
    private int fuelTankCapacity;
    private boolean isAvailableForTestDrive;

    public Car(String model, String color, int seatingCapacity, int
fuelTankCapacity, boolean isAvailableForTestDrive) {
        this.model = model;
        this.color = color;
```

```java
        this.seatingCapacity = seatingCapacity;
        this.fuelTankCapacity = fuelTankCapacity;
        this.isAvailableForTestDrive = isAvailableForTestDrive;
    }

    @Override
    public void start() {
        System.out.println("Car is started and moving");
    }

    @Override
    public void stop() {
        System.out.println("Car is stopped");
    }

    // Getters
    public String getModel() {
      return model;
    }

    public String getColor() {
        return color;
    }

    public int getSeatingCapacity() {
        return seatingCapacity;
    }

    public int getFuelTankCapacity() {
        return fuelTankCapacity;
    }

    public boolean isAvailableForTestDrive() {
        return isAvailableForTestDrive;
    }
}

public class Bike extends Vehicle {
    private String model;
    private String color;
    private int seatingCapacity;
    private int fuelTankCapacity;
    private boolean isAvailableForTestDrive;

    public Bike(String model, String color, int fuelTankCapacity) {
        this.model = model;
        this.color = color;
        this.fuelTankCapacity = fuelTankCapacity;
        this.isAvailableForTestDrive = false;
        this.seatingCapacity = 2;
    }

    @Override
    public void start() {
        System.out.println("Bike is started and moving");
    }

    @Override
    public void stop() {
        System.out.println("Bike is stopped");
    }

    // Getters
    public String getModel() {
        return model;
    }

    public String getColor() {
        return color;
    }

```

```
    public int getSeatingCapacity() {
        return seatingCapacity;
    }

    public int getFuelTankCapacity() {
        return fuelTankCapacity;
    }

    public boolean isAvailableForTestDrive() {
        return isAvailableForTestDrive;
    }
}
```

```
public class VehicleFactory {

    public static Vehicle getVehicle(String type) {
        if (type.equals("car")) {
            return new Car("Toyota", "Red", 5, 60, true);
        } else if (type.equals("bike")) {
            return new Bike("Yamaha", "Black", 60);
        } else {
            return null; // THE PROBLEM
        }
    }
}
public class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("##### Null Object Pattern: Problem Demo #####");

        Vehicle car = VehicleFactory.getVehicle("car");
        printVehicleDetails(car);
        testDrive(car);

        Vehicle bike = VehicleFactory.getVehicle("bike");
        printVehicleDetails(bike);
        testDrive(car);

        // Saved by NULL Check in printVehicleDetails and testDrive methods
        // Without NULL Check, it will not throw NullPointerException or ClassCastException
        Vehicle nullVehicle = VehicleFactory.getVehicle("null");
        printVehicleDetails(nullVehicle);
        testDrive(nullVehicle);
    }

    private static void printVehicleDetails(Vehicle vehicle) {
        if (vehicle != null) { // THE PROBLEM
            if (vehicle instanceof Car car) {
              System.out.print("\n[+] Vehicle Details: ");
                System.out.println(car.getClass().getSimpleName() + "[Model=" + car.getModel()
                        + ", Color=" + car.getColor() + ", Seating Capacity=" + car.getSeatingCapacity()
                        + ", Fuel Tank Capacity=" + car.getFuelTankCapacity() + "]");

            }
            if (vehicle instanceof Bike bike) { // THE PROBLEM
                System.out.print("\n[+] Vehicle Details: ");
                System.out.println(bike.getClass().getSimpleName() + "[Model=" + bike.getModel()
                        + ", Color=" + bike.getColor() + ", Fuel Tank Capacity=" + bike.getFuelTankCapacity() + "]");
            }
        }
    }

    private static void testDrive(Vehicle vehicle) {
        if (vehicle != null) { // THE PROBLEM
```

```
            vehicle.start();
            vehicle.stop();
        }
    }
}
```

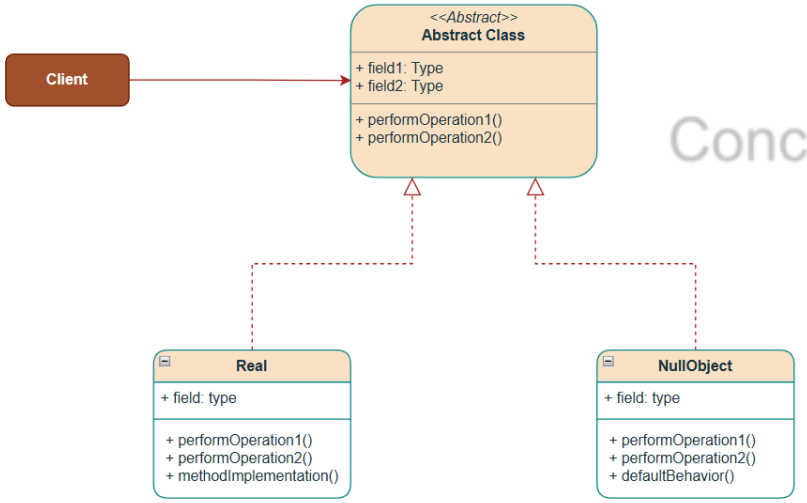
NULL Object Design Pattern

> **The NULL Object Pattern is a behavioral design pattern that uses polymorphism to eliminate null checks. Instead of returning NULL and adding NULL checks wherever necessary, we return a special object called NULL OBJECT that implements the expected interface(or extends an abstract class) but does nothing (or provides default behavior).**

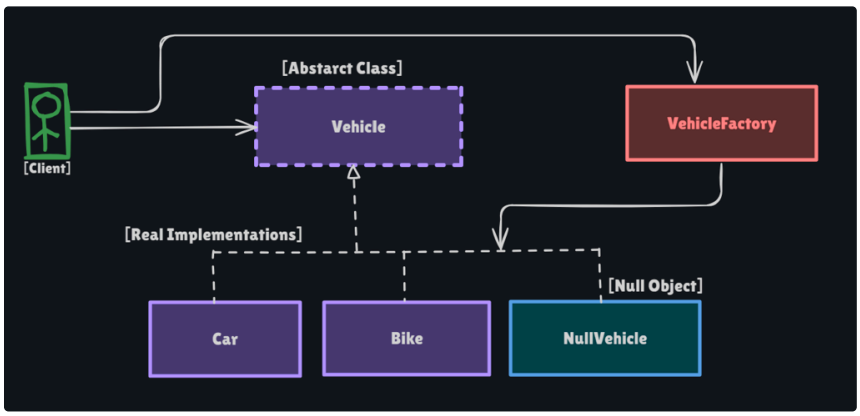
Key Points

- Instead of returning null, return an instance of a Null Object. A NULL Object replaces a NULL return type.
- This leads to clean code without redundant NULL checks everywhere.
- NULL Object reflects do Nothing or contains a Default behaviour.

Class Diagram



Structure of NULL Object Pattern



1. **Interface**: `Vehicle` defines the contract to be implemented by `Real` & `NullObject` classes provide respective behaviour.
2. **Real Implementations**: `Car` and `Bike` do the actual work.
3. **NULL Object**: `NullVehicle` implements the interface but does nothing. Provides default properties & behaviour.
4. **Factory**: `VehicleFactory` receives `vehicle-type` as client input to create and return a `Real` specific implementation of the `Vehicle` object. If the client provides a non-existent vehicle type, the factory returns a `NullVehicle` instance.

Implementation

The `Vehicle`, `Car`, and `Bike` classes remain unchanged as we add the `NullVehicle` class to the hierarchy, allowing the Factory to return a `NullObject` with default behaviour when the requested vehicle type doesn't match.

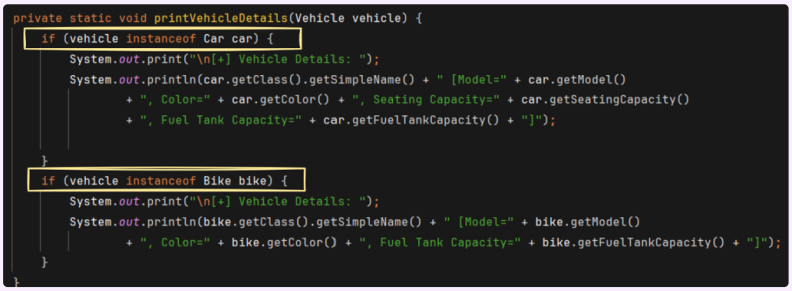
```java
public class NullVehicle extends Vehicle {
    private final String model;
    private final String color;
    private final int seatingCapacity;
    private final int fuelTankCapacity;
    private final boolean isAvailableForTestDrive;

    public NullVehicle() {
        this.model = "Default";
        this.color = "Default";
        this.seatingCapacity = 0;
        this.fuelTankCapacity = 0;
        this.isAvailableForTestDrive = false;
    }

    @Override
    public void start() {
        // Do nothing - silent Vehicle
        System.out.print("\n[-] Null Vehicle: start() - do nothing");
    }

    @Override
    public void stop() {
        // Do nothing - silent Vehicle
        System.out.println("\n[-] Null Vehicle: stop() - do nothing");
    }

    // Getters
    public int getSeatingCapacity() {
        return seatingCapacity;
    }

    public int getFuelTankCapacity() {
        return fuelTankCapacity;
    }

    public boolean isAvailableForTestDrive() {
        return isAvailableForTestDrive;
    }
}
```



```java
public class VehicleFactory {

    public static Vehicle getVehicle(String type) {
        if (type.equals("car")) {
            return new Car("Toyota", "Red", 5, 60, true);
```

```
        } else if (type.equals("bike")) {
            return new Bike("Yamaha", "Black", 60);
        } else {
            return new NullVehicle(); // THE SOLUTION
        }
    }
}
```

```
public class SolutionDemo {
    public static void main(String[] args) {
        System.out.println("\n##### Null Object Pattern: Solution Demo #####");

        Vehicle car = VehicleFactory.getVehicle("car");
        printVehicleDetails(car);
        testDrive(car);

        Vehicle bike = VehicleFactory.getVehicle("bike");
        printVehicleDetails(bike);
        testDrive(car);

        // Saved by NULL Check in printVehicleDetails and testDrive methods
        // Without NULL Check, it will not throw NullPointerException or ClassCastException
        Vehicle nullVehicle = VehicleFactory.getVehicle("null");
        printVehicleDetails(nullVehicle);
        testDrive(nullVehicle);
    }

    private static void printVehicleDetails(Vehicle vehicle) {
            if (vehicle instanceof Car car) {
                System.out.print("\n[+] Vehicle Details: ");
                System.out.println(car.getClass().getSimpleName() + " [Model=" + car.getModel()
                        + ", Color=" + car.getColor() + ", Seating Capacity=" + car.getSeatingCapacity()
                        + ", Fuel Tank Capacity=" + car.getFuelTankCapacity() + "]");

            }
            if (vehicle instanceof Bike bike) {
                System.out.print("\n[+] Vehicle Details: ");
                System.out.println(bike.getClass().getSimpleName() + " [Model=" + bike.getModel()
                        + ", Color=" + bike.getColor() + ", Fuel Tank Capacity=" + bike.getFuelTankCapacity() + "]");
            }
    }

    private static void testDrive(Vehicle vehicle) {
            vehicle.start();
            vehicle.stop();
    }
}
```

This is a Java pattern-matching feature introduced in Java 16.

This line does two things at once:

1. It checks if the `vehicle` object is an instance of the `Car` class.
2. If it is, it automatically casts the `vehicle` to the `Car` type and assigns it to a new variable named `car` that can be used within the if block.

Output

```
##### Null Object Pattern: Solution Demo #####

[+] Vehicle Details: Car [Model=Toyota, Color=Red, Seating Capacity=5, Fuel Tank Capacity=60]
Car is started and moving
Car is stopped

[+] Vehicle Details: Bike [Model=Yamaha, Color=Black, Fuel Tank Capacity=60]
Car is started and moving
Car is stopped

[-] Null Vehicle: start() - do nothing
[-] Null Vehicle: stop() - do nothing

Process finished with exit code 0
```

Benefits

- Makes code cleaner by eliminating repetitive null checks.
- Reduces `NullPointerException` risks.
- Increases code readability.

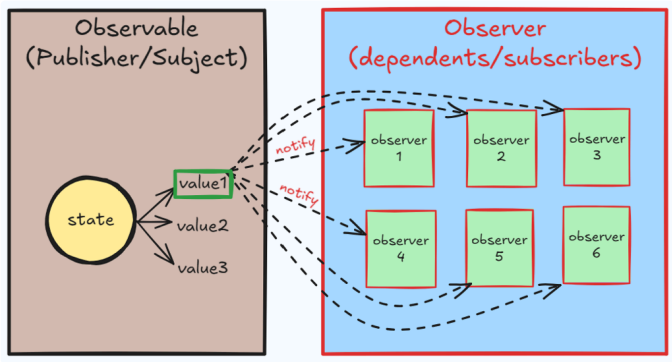
Observer Pattern

Resources

- Video link → 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- Video link → 3. Observer Design Pattern Explanation (Hindi) | Design Interview Question | LLD System Design

Definition

> ***The Observer Pattern is a behavioral design pattern where an object (aka the "subject" or "observable" or “publisher”) maintains a list of dependents (called "observers") and automatically notifies them whenever there is a change in its state. The pattern also allows addition and removal of observers at runtime.***

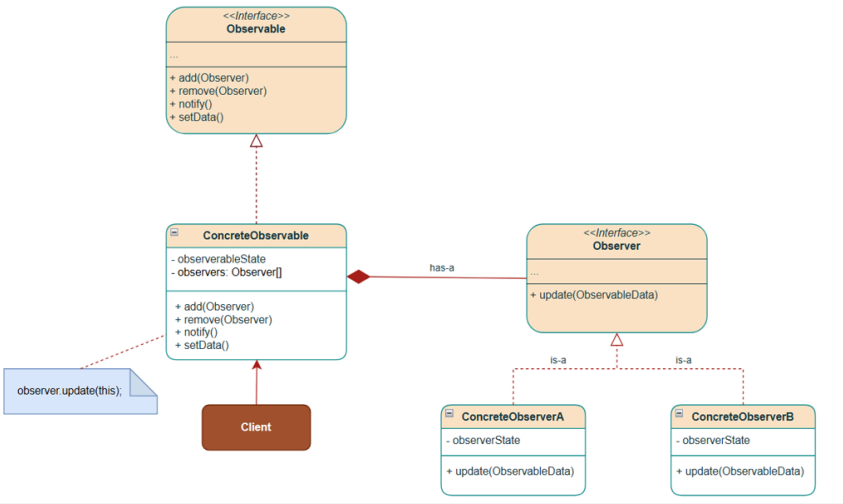


Real-life Examples

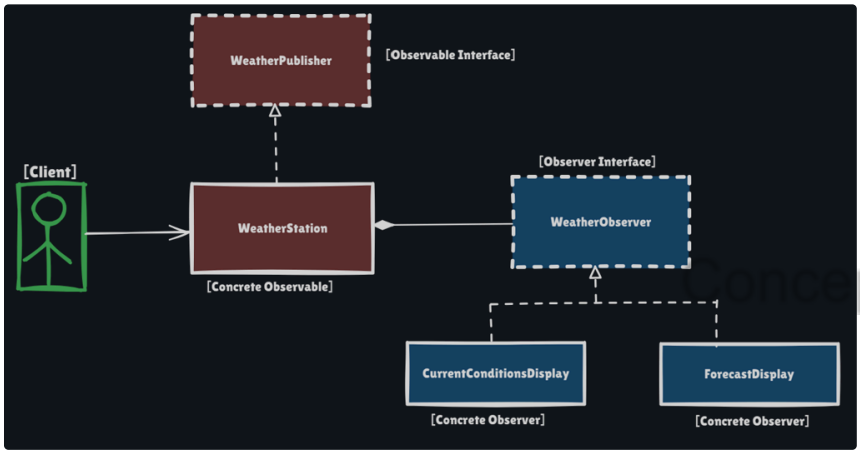
Real-life examples of the Observer pattern include:

- **Weather Applications:** Where multiple devices receive updates from a weather station.
- **Social Media Feeds:** When we follow someone on Instagram, Facebook, or Twitter, we become observer of their profile. When they post new content, we are automatically notified.
- **Subscription Services:** YouTube subscriptions, where viewers are notified of new videos, or Content magazine/newspaper/newsletter subscriptions, where publishers send new issues to subscribers.
- **Stock Market Trackers:** When the price of a stock (or state) changes, the stock's market system (the observable) sends out notifications to all interested investors (the observers).

Class Diagram



Structure of the Observer Pattern



Let's understand the Structure of the Observer Pattern using the Weather Station example:

- **Observable Interface (**or Subject Interface, i.e., `WeatherObservable` **)**
  - Defines methods for adding, removing, and notifying observers.
  - The weather station implements this interface.
- **Observer Interface (** `WeatherObserver` **)**
  - Defines the `update()` method that all concrete observers must implement.
  - Called by the observable when there is a change in its state.
- **Concrete Observable (or Concrete Subject, i.e.,** `WeatherStation` **)**
  - Maintains a list of observers.
  - Holds the weather data, i.e., **observable data** (temperature, humidity, pressure).
  - Notifies all observers when measurements(state) change.
- **Concrete Observers (** `ForecastDisplay` **&** `CurrentConditionsDisplay` **)**
  - Each display has different behavior when updated.
  - `CurrentConditionsDisplay` : Shows current weather on gadgets like TV or mobile.

- `ForecastDisplay`: Predicts weather based on pressure changes.

Implementation

1. Example: Weather Station

```java
// Observable(Subject) interface
// Defines methods for managing observers and notifying them of changes
public interface WeatherObservable {

    void addObserver(WeatherObserver observer);

    void removeObserver(WeatherObserver observer);

    void notifyObservers();

    void setWeatherReadings(float temperature, float humidity, float pressure);
}
```

```java
// Concrete Observable (Subject)
// WeatherStation - the concrete observable class that holds weather data
public class WeatherStation implements WeatherObservable {
    // List of observers registered for updates
    private final List<WeatherObserver> observers;
    // Observable Data
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherStation() {
        observers = new ArrayList<>();
    }

    @Override
    public void addObserver(WeatherObserver observer) {
        observers.add(observer);
        System.out.println("[+] Observer registered: " + observer.getClass().getSimpleName());
    }

    @Override
    public void removeObserver(WeatherObserver observer) {
        observers.remove(observer);
        System.out.println("[-] Observer removed: " + observer.getClass().getSimpleName());
    }

    @Override
    public void notifyObservers() {
        for (WeatherObserver observer : observers) {
            // Notify each observer about the change in weather data(state)
            observer.update(); // Observer will update its state based on the new data and respond accordingly
        }
    }

    // Method to update weather measurements
    public void setWeatherReadings(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        notifyObservers();
    }
```

```
42
43      // Getters for observers to access weather data
44      public float getTemperature() {
45          return temperature;
46      }
47
48      public float getHumidity() {
49          return humidity;
50      }
51
52      public float getPressure() {
53          return pressure;
54      }
55
56      @Override
57      public String toString() {
58          return "WeatherStation{" +
59                  "temperature=" + temperature +
60                  ", humidity=" + humidity +
61                  ", pressure=" + pressure +
62                  '}';
63      }
64  }
```

```
1  // Observer interface - defines the update method
2  // Concrete observers implement this interface to update their state
3  // and respond to changes in its OWN way
4  public interface WeatherObserver {
5      void update();
6  }
```

```
1  // Concrete Observer 1 - Current Conditions Display (on TV or Mobile)
2  public class CurrentConditionsDisplay implements WeatherObserver {
3      private final WeatherObservable weatherStation;
4
5      public CurrentConditionsDisplay(WeatherObservable weatherStation)
   {
6          this.weatherStation = weatherStation;
7          weatherStation.addObserver(this);
8      }
9
10     // CurrentConditionsDisplay implements the update method in its
   own way
11     @Override
12     public void update() {
13         System.out.println("Saving weather data... ");
14         display();
15     }
16
17     // Display the current weather conditions
18     public void display() {
19         System.out.println("Current Weather Conditions: " +
   weatherStation.toString());
20     }
21 }
```



```
1  // Concrete Observer 4 - Forecast Display - Predicts weather based on
   pressure changes
2  public class ForecastDisplay implements WeatherObserver {
3      private final WeatherObservable weatherStation;
4
5      public ForecastDisplay(WeatherObservable weatherStation) {
6          this.weatherStation = weatherStation;
7          weatherStation.addObserver(this);
8      }
9
10     // ForecastDisplay implements the update method in its own way
11     @Override
12     public void update() {
```

```
        System.out.println("Updating weather data to do some analytics: " + weatherStation.toString());
        display();
    }

    // Display the forecast based on the current pressure
    public void display() {
        System.out.println("Forecast Details: Displaying information about Rain, " +
                "Temperature Trends, Significant Weather Events and other phenomemnon...");
    }
}

```

```
// Client code to demonstrate the Observer Pattern
public class WeatherStationApp {
    public static void main(String[] args) {
        System.out.println("###### State Design Pattern ######");
        // Create the weather station (observable/subject)
        WeatherObservable weatherStation = new WeatherStation();

        // Create displays (observers)
        CurrentConditionsDisplay currentDisplay = new CurrentConditionsDisplay(weatherStation);
        ForecastDisplay forecastDisplay = new ForecastDisplay(weatherStation);

        System.out.println("===>>> Initial Weather Update");
        weatherStation.setWeatherReadings(80, 65, 30.4f);

        System.out.println("===>>> Second Weather Update");
        weatherStation.setWeatherReadings(82, 70, 29.2f);

        // Remove forecast display
        weatherStation.removeObserver(forecastDisplay);

        System.out.println("===>>> Third Weather Update");
        weatherStation.setWeatherReadings(70, 21, 29.2f);
        // Forecast display will not be notified
    }
}
```


Output

```
###### State Design Pattern ######
[+] Observer registered: CurrentConditionsDisplay
[+] Observer registered: ForecastDisplay
===>>> Initial Weather Update
Saving weather data...
Current Weather Conditions: WeatherStation{temperature=80.0, humidity=65.0, pressure=30.4}
Updating weather data to do some analytics: WeatherStation{temperature=80.0, humidity=65.0, pressure=30.4}
Forecast Details: Displaying information about Rain, Temperature Trends, Significant Weather Events and other phenomemnon...
===>>> Second Weather Update
Saving weather data...
Current Weather Conditions: WeatherStation{temperature=82.0, humidity=70.0, pressure=29.2}
Updating weather data to do some analytics: WeatherStation{temperature=82.0, humidity=70.0, pressure=29.2}
Forecast Details: Displaying information about Rain, Temperature Trends, Significant Weather Events and other phenomemnon...
[-] Observer removed: ForecastDisplay
===>>> Third Weather Update
Saving weather data...
Current Weather Conditions: WeatherStation{temperature=70.0, humidity=21.0, pressure=29.2}

Process finished with exit code 0
```

2. Example: E-commerce “Notify Me” feature

```
// Observable interface
public interface StockAvailabilityObservable {
    void addStockObserver(StockNotificationObserver observer);

    void removeStockObserver(StockNotificationObserver observer);

    void notifyStockObservers();

```

```
    boolean purchase(int quantity);

    void restock(int quantity);
}
```

```
// Concrete Observable
public class IphoneProductObservable implements StockAvailabilityObservable {
    private final String productId;
    private final String productName;
    private final double price;
    private final List<StockNotificationObserver> stockObservers;
    private int stockQuantity;

    public IphoneProductObservable(String productId, String productName, double price, int stockQuantity) {
        this.productId = productId;
        this.productName = productName;
        this.price = price;
        this.stockQuantity = stockQuantity;
        this.stockObservers = new ArrayList<>();
    }

    @Override
    public void addStockObserver(StockNotificationObserver observer) {
        stockObservers.add(observer);
        System.out.println("[+]" + observer.getUserId() + " subscribed for notifications on " + productName);

    }

    @Override
    public void removeStockObserver(StockNotificationObserver observer) {
        stockObservers.remove(observer);
        System.out.println("[-]" + observer.getUserId() + " unsubscribed for notifications on " + productName);
    }

    @Override
    public void notifyStockObservers() {
        if (stockQuantity > 0 && !stockObservers.isEmpty()) {
            System.out.println("Notifying " + stockObservers.size() + " subscribers... ");

            // Create a copy to avoid concurrent modification
            List<StockNotificationObserver> observersToNotify = new ArrayList<>(stockObservers);

            for (StockNotificationObserver observer : observersToNotify) {
                observer.update();
            }
        }
    }

    // Method to restock items
    @Override
    public void restock(int quantity) {
        boolean wasOutOfStock = (stockQuantity == 0);
        stockQuantity += quantity;
        System.out.println("RESTOCKED: " + productName + " - Added " + quantity + " items " + " | " + "Current stock: " + stockQuantity);
        // Only notify if product was previously out of stock
        if (wasOutOfStock && stockQuantity > 0) {
            notifyStockObservers();
        }
    }

    // Method to purchase items
    @Override
```

```
    public boolean purchase(int quantity) {
        if (stockQuantity >= quantity) {
            stockQuantity -= quantity;
            System.out.println("PURCHASE SUCCESS: " + quantity + " units of " + productName + " | " + "Remaining stock: " + stockQuantity);
            return true;
        } else {
            System.out.println("PURCHASE FAILED: " + productName + " is out of stock! | " + "Available Quantity: " + stockQuantity);
            return false;
        }
    }

    // Getters
    public String getProductId() {
        return productId;
    }

    public String getProductName() {
        return productName;
    }

    public double getPrice() {
        return price;
    }

    public int getStockQuantity() {
        return stockQuantity;
    }
}

```

```
// Observer interface for stock availability notifications
public interface StockNotificationObserver {
    void update();

    String getNotificationMethod();

    String getUserId();
}
```



```
// Concrete observer for email notifications
public class EmailNotificationObserver implements StockNotificationObserver {
    private final String userId;
    private final String emailAddress;

    public EmailNotificationObserver(String userId, String emailAddress) {
        this.userId = userId;
        this.emailAddress = emailAddress;
    }

    @Override
    public void update() {
        sendEmail();
    }

    private void sendEmail() {
        System.out.println("!! EMAIL SENT to: " + emailAddress + " - " + "Product is back in stock! Hurry Up!!");
    }

    @Override
    public String getNotificationMethod() {
        return "Email";
    }

    @Override
    public String getUserId() {
```

```
        return userId;
    }
}
```

```
// Concrete observer for push notifications
public class PushNotificationObserver implements StockNotificationObserver {
    private final String userId;
    private final String deviceToken;

    public PushNotificationObserver(String userId, String deviceToken) {
        this.userId = userId;
        this.deviceToken = deviceToken;
    }

    @Override
    public void update() {
        sendPushNotification();
    }

    private void sendPushNotification() {
        System.out.println("!! PUSH NOTIFICATION SENT to: " + deviceToken + " - " + "Product is back in stock! Hurry Up!!");
    }

    @Override
    public String getNotificationMethod() {
        return "Push Notification";
    }

    @Override
    public String getUserId() {
        return userId;
    }
}
```



```
public class ECommerceStockNotificationApp {
        System.out.println("-----------------------------------------------------------------------");
        System.out.println("###### E-commerce Store - Stock Availability Notification Feature Demo ######");

        // Create an iPhone product - stock available = 10 units
        StockAvailabilityObservable iphoneProduct = new IphoneProductObservable("ip15", "iphone 15", 1250, 10);

        // Create observers
        StockNotificationObserver John_PUSH = new PushNotificationObserver("John123", "JohnDeviceP1");
        StockNotificationObserver Katy_PUSH = new PushNotificationObserver("Katy678", "KatyDeviceP2");
        StockNotificationObserver Jane_EMAIL = new EmailNotificationObserver("Jane783", "jane783@gmail.com");
        StockNotificationObserver George_EMAIL = new EmailNotificationObserver("George993", "george993@gmail.com");

        // Black Friday Sale - Purchase all 10 units
        iphoneProduct.purchase(10);

        // Stock unavailability leads to users subscribing to notifications
        boolean success = iphoneProduct.purchase(1); // Failed purchase
        if (!success) {
            // Register observers - John, Katy, Jane, George subscribe for notifications upon stock availability
            iphoneProduct.addStockObserver(John_PUSH); // John
            iphoneProduct.addStockObserver(Katy_PUSH); // Katy
            iphoneProduct.addStockObserver(Jane_EMAIL); // Jane
            iphoneProduct.addStockObserver(George_EMAIL); // George
```

```
        }

        // Restock 20 units of iPhone 15
        iphoneProduct.restock(20); // All 4 observers are notified

        // Users purchase upon receiving notifications
        iphoneProduct.purchase(1); // John purchases 1 unit
        iphoneProduct.purchase(1); // Katy purchases 1 unit

        // John & Katy unsubscribe from notifications
        iphoneProduct.removeStockObserver(John_PUSH);
        iphoneProduct.removeStockObserver(Katy_PUSH);

        // NYE Sale - All 18 units sold
        iphoneProduct.purchase(18);
        iphoneProduct.restock(5); // Only Jane & George are notified

        iphoneProduct.purchase(1); // Jane purchases 1 unit
        iphoneProduct.purchase(1); // George purchases 1 unit

        // Jane & George unsubscribe from notifications
        iphoneProduct.removeStockObserver(Jane_EMAIL);
        iphoneProduct.removeStockObserver(George_EMAIL);
    }
}
```

Output

```
--------------------------------------------------------------------------------
###### E-commerce Store - Stock Availability Notification Feature Demo ######
PURCHASE SUCCESS: 10 units of iphone 15 | Remaining stock: 0
PURCHASE FAILED: iphone 15 is out of stock! | Available Quantity: 0
[+]John123 subscribed for notifications on iphone 15
[+]Katy678 subscribed for notifications on iphone 15
[+]Jane783 subscribed for notifications on iphone 15
[+]George993 subscribed for notifications on iphone 15
RESTOCKED: iphone 15 - Added 20 items  | Current stock: 20
Notifying 4 subscribers...
!! PUSH NOTIFICATION SENT to: JohnDeviceP1 - Product is back in stock! Hurry Up!!
!! PUSH NOTIFICATION SENT to: KatyDeviceP2 - Product is back in stock! Hurry Up!!
!! EMAIL SENT to: jane783@gmail.com - Product is back in stock! Hurry Up!!
!! EMAIL SENT to: george993@gmail.com - Product is back in stock! Hurry Up!!
PURCHASE SUCCESS: 1 units of iphone 15 | Remaining stock: 19
PURCHASE SUCCESS: 1 units of iphone 15 | Remaining stock: 18
[-]John123 unsubscribed for notifications on iphone 15
[-]Katy678 unsubscribed for notifications on iphone 15
PURCHASE SUCCESS: 18 units of iphone 15 | Remaining stock: 0
RESTOCKED: iphone 15 - Added 5 items  | Current stock: 5
Notifying 2 subscribers...
!! EMAIL SENT to: jane783@gmail.com - Product is back in stock! Hurry Up!!
!! EMAIL SENT to: george993@gmail.com - Product is back in stock! Hurry Up!!
PURCHASE SUCCESS: 1 units of iphone 15 | Remaining stock: 4
PURCHASE SUCCESS: 1 units of iphone 15 | Remaining stock: 3
[-]Jane783 unsubscribed for notifications on iphone 15
[-]George993 unsubscribed for notifications on iphone 15
```

Vending Machine Using State Pattern Pattern

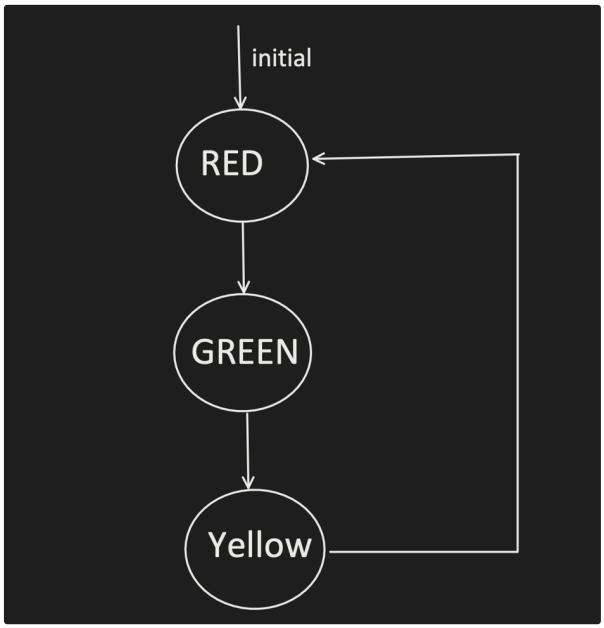
Resources

- Video → 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- Video → 16. Design Vending Machine (Hindi) | LLD of Vending Machine | State Design Pattern | LLD question

Definition

> **The State Pattern allows an object to change its behavior dynamically at runtime whenever there is a change in its internal state.**

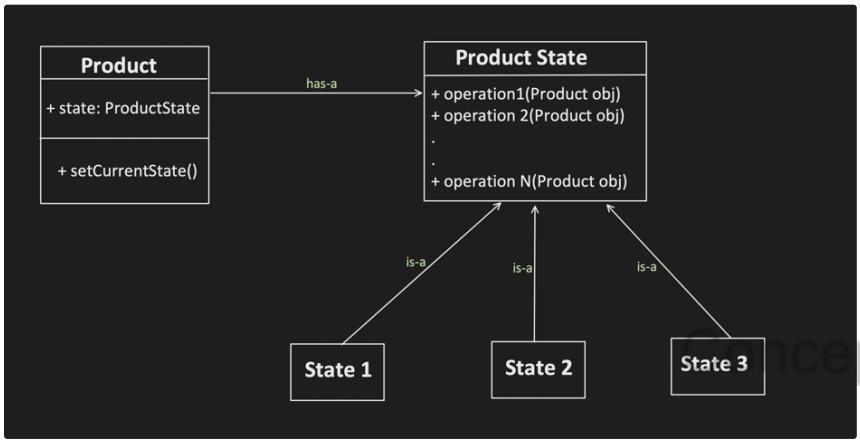
Real Life Example: Traffic Signal



State	Operations
Red State (Stop)	action() -> Make Signal Red
Green State (Go)	action() ->Make Signal Green
Yellow State (Slow Down)	action() ->Make Signal Yellow

This type of problems, where Object change the state after performing certain operation can be solved through : State Design Pattern

State Design Pattern UML:



Lets Code this UML for Traffic Signal:

```
public interface TrafficLightState {
    void action(TrafficLight signal);
}

```

```
public class RedState implements TrafficLightState {

   @Override
    public void action(TrafficLight signal) {
        //STOP behavior
        signal.setState(new GreenState()); // next state
    }
}


public class GreenState implements TrafficLightState {

  @Override
    public void action(TrafficLight signal) {
        //GO behavior
        signal.setState(new YellowState()); // next state
    }
```

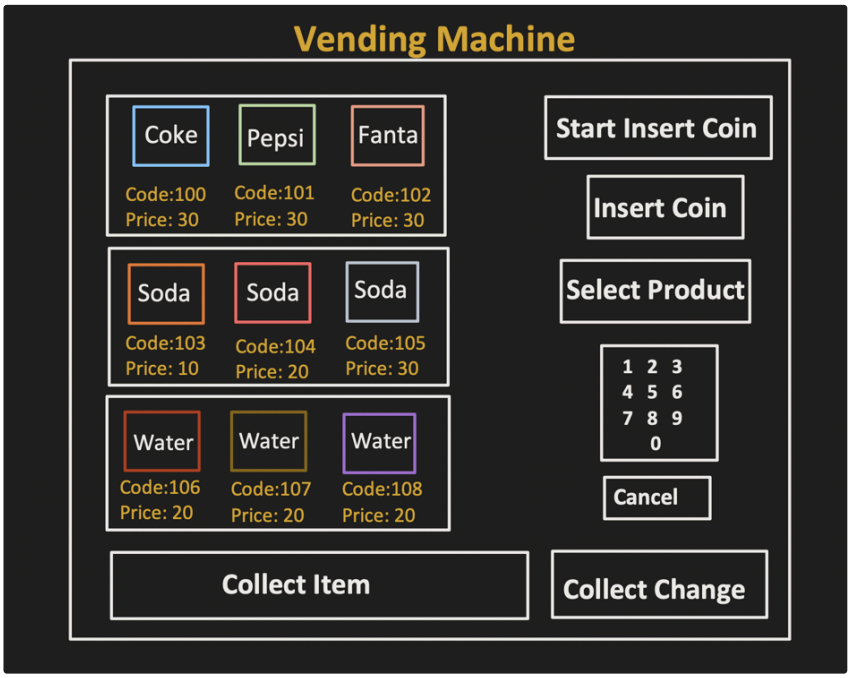
```
}


public class YellowState implements TrafficLightState {

  @Override
    public void action(TrafficLight signal) {
        //Slow Down behavior
        signal.setState(new RedState()); // next state
    }
}


```

```
public class TrafficLight {
    private TrafficLightState state;

    public TrafficLight() {
        this.state = new RedState(); // initial state
    }

    public void setState(TrafficLightState state) {
        this.state = state;
    }

    public void change() {
        state.action(this);
    }
}

```

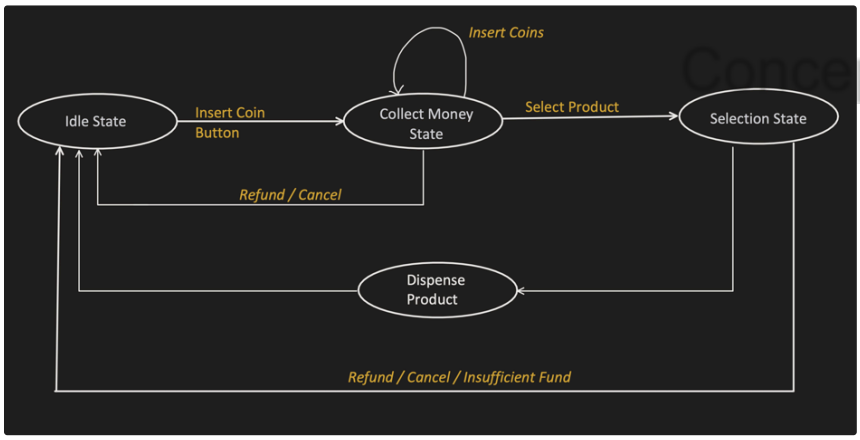


```
public class Main {
    public static void main(String[] args) {
        TrafficLight trafficLight = new TrafficLight (); //initial
signal state is RED

        trafficLight.change(); // RED → GREEN
        trafficLight.change(); // GREEN → YELLOW
        trafficLight.change(); // YELLOW → RED
    }
}

```

Real Life Example: Vending Machine



Understanding the working of a Vending Machine



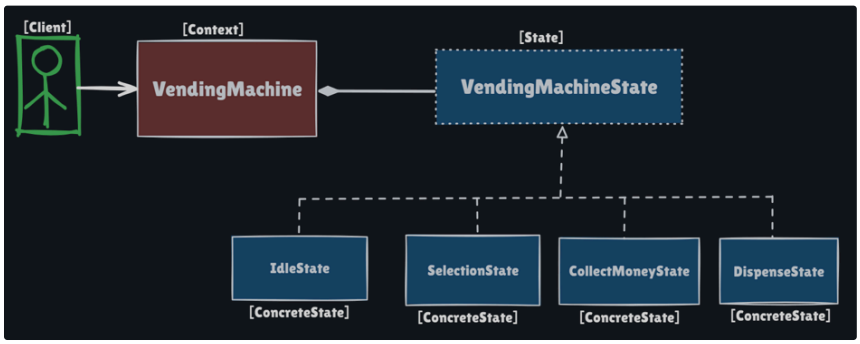
Different States and Operations

Example: Vending Machine

State	Operations
IdleState	• Press Insert Coin button
HasMoney State	• Insert Coin • Select Product • Cancel / Refund
Selection State	• Choose Product • Return change

	• Cancel/Refund full amount
Dispense Product State	• Dispense Product

Vending Machine representation using State Pattern



1. **State Interface(e.g., `VendingMachineState`):** Declares common functions that all states must implement.
2. **Concrete States(e.g., `IdleState`, `SelectionState`, `hasMoney`, `DispenseState`):** Each class implements the state interface behaviors(operations) differently depending on the current state of the vending machine, and an exception is thrown for operations that do not apply to the current state.
3. **Context Class** (e.g., `VendingMachine`): Maintains a reference to the current state. Holds all possible states as objects. Delegates call to the current state object.
4. **Client(`VendingMachineAppDemo`):** Interacts with Context Class (`VendingMachine`) and expects appropriate behavior as per changes in the state of the object.

Implementation(e.g., Vending Machine)

```
1  public abstract class State {
2
3      public void clickOnInsertCoinButton(VendingMachine machine) throws
   Exception {
4          // by default nothing happens
5      }
6
7      public void clickOnStartProductSelectionButton(VendingMachine
   machine) throws Exception {
8          // by default nothing happens
9      }
10
11     public void insertCoin(VendingMachine machine, Coin coin) throws
   Exception {
12         // by default nothing happens
13     }
14
15     public void chooseProduct(VendingMachine machine, int codeNumber)
   throws Exception {
16         // by default nothing happens
17     }
18
19     public int getChange(int returnChangeMoney) throws Exception {
20         // by default nothing happens
21         return 0;
22     }
23
24     public Item dispenseProduct(VendingMachine machine, int
   codeNumber) throws Exception {
25         // by default nothing happens
```

```
        return null;
    }

    public List<Coin> refundFullMoney(VendingMachine machine) throws
Exception {
        // by default nothing happens
        return null;
    }

    public void updateInventory(VendingMachine machine, Item item, int
codeNumber) throws Exception {
        // by default nothing happens
    }
}
```

```
public class IdleState extends State {

    public IdleState(){
        System.out.println("Currently Vending machine is in
IdleState");
    }

    public IdleState(VendingMachine machine){
        System.out.println("Currently Vending machine is in
IdleState");
        machine.setCoinList(new ArrayList<>());
    }

    @Override
    public void clickOnInsertCoinButton(VendingMachine machine) throws
Exception{
        machine.setVendingMachineState(new HasMoneyState());
    }

    @Override
    public void updateInventory(VendingMachine machine, Item item, int
codeNumber) throws Exception {
        machine.getInventory().addItem(item, codeNumber);
    }
}
```



```
public class HasMoneyState extends State {

    public HasMoneyState(){
        System.out.println("Currently Vending machine is in
HasMoneyState");
    }

    @Override
    public void clickOnStartProductSelectionButton(VendingMachine
machine) throws Exception {
        machine.setVendingMachineState(new SelectionState());
    }

    @Override
    public void insertCoin(VendingMachine machine, Coin coin) throws
Exception {
        System.out.println("Accepted the coin");
        machine.getCoinList().add(coin);
    }

    @Override
    public List<Coin> refundFullMoney(VendingMachine machine) throws
Exception {
        System.out.println("Returned the full amount back in the Coin
Dispense Tray");
        machine.setVendingMachineState(new IdleState(machine));
        return machine.getCoinList();
```

```
    }
}
```

```
public class SelectionState extends State {

    public SelectionState(){
        System.out.println("Currently Vending machine is in
SelectionState");
    }

    @Override
    public void chooseProduct(VendingMachine machine, int codeNumber)
throws Exception{

        //1. get item of this codeNumber
        Item item = machine.getInventory().getItem(codeNumber);

        //2. total amount paid by User
        int paidByUser = 0;
        for(Coin coin : machine.getCoinList()){
            paidByUser = paidByUser + coin.value;
        }

        //3. compare product price and amount paid by user
        if(paidByUser < item.getPrice()) {
            System.out.println("Insufficient Amount, Product you
selected is for price: " + item.getPrice() + " and you paid: " +
paidByUser);
            refundFullMoney(machine);
            throw new Exception("insufficient amount");
        }
        else if(paidByUser >= item.getPrice()) {

            if(paidByUser > item.getPrice()) {
                getChange(paidByUser-item.getPrice());
            }
            machine.setVendingMachineState(new DispenseState(machine,
codeNumber));
        }
    }

    @Override
    public int getChange(int returnExtraMoney) throws Exception{
        //actual logic should be to return COINs in the dispense tray,
but for simplicity i am just returning the amount to be refunded
        System.out.println("Returned the change in the Coin Dispense
Tray: " + returnExtraMoney);
        return returnExtraMoney;
    }

    @Override
    public List<Coin> refundFullMoney(VendingMachine machine) throws
Exception{
        System.out.println("Returned the full amount back in the Coin
Dispense Tray");
        machine.setVendingMachineState(new IdleState(machine));
        return machine.getCoinList();
    }
}
```



```
public class DispenseState extends State {

    DispenseState(VendingMachine machine, int codeNumber) throws
Exception{
        System.out.println("Currently Vending machine is in
DispenseState");
        dispenseProduct(machine, codeNumber);
    }

    @Override
```

```
    public Item dispenseProduct(VendingMachine machine, int
codeNumber) throws Exception{
        System.out.println("Product has been dispensed");
        Item item = machine.getInventory().getItem(codeNumber);
        machine.getInventory().updateSoldOutItem(codeNumber);
        machine.setVendingMachineState(new IdleState(machine));
        return item;
    }
}
```

```
public class VendingMachine {

    private State vendingMachineState;
    private Inventory inventory;
    private List<Coin> coinList;

    public VendingMachine(){
        vendingMachineState = new IdleState();
        inventory = new Inventory(10);
        coinList = new ArrayList<>();
    }

    public State getVendingMachineState() {
        return vendingMachineState;
    }

    public void setVendingMachineState(State vendingMachineState) {
        this.vendingMachineState = vendingMachineState;
    }

    public Inventory getInventory() {
        return inventory;
    }

    public void setInventory(Inventory inventory) {
        this.inventory = inventory;
    }

    public List<Coin> getCoinList() {
        return coinList;
    }

    public void setCoinList(List<Coin> coinList) {
        this.coinList = coinList;
    }
}
```



```
public enum Coin {

    PENNY(1),
    NICKEL(5),
    DIME(10),
    QUARTER(25);

    public int value;

    Coin(int value) {
        this.value = value;
    }
}
```

```
public class Inventory {

```

```
    ItemShelf[] inventory = null;

    Inventory(int itemCount) {
        inventory = new ItemShelf[itemCount];
        initialEmptyInventory();
    }

    public ItemShelf[] getInventory() {
        return inventory;
    }

    public void setInventory(ItemShelf[] inventory) {
        this.inventory = inventory;
    }

    public void initialEmptyInventory() {
        int startCode = 101;
        for (int i = 0; i < inventory.length; i++) {
            ItemShelf space = new ItemShelf();
            space.setCode(startCode);
            space.setSoldOut(true);
            inventory[i]= space;
            startCode++;
        }
    }

    public void addItem(Item item, int codeNumber) throws Exception {

        for (ItemShelf itemShelf : inventory) {
            if (itemShelf.code == codeNumber) {
                if (itemShelf.isSoldOut()) {
                    itemShelf.item = item;
                    itemShelf.setSoldOut(false);
                } else {
                    throw new Exception("already item is present, you
can not add item here");
                }
            }
        }
    }

    public Item getItem(int codeNumber) throws Exception {

        for (ItemShelf itemShelf : inventory) {
            if (itemShelf.code == codeNumber) {
                if (itemShelf.isSoldOut()) {
                    throw new Exception("item already sold out");
                } else {

                    return itemShelf.item;
                }
            }
        }
        throw new Exception("Invalid Code");
    }

    public void updateSoldOutItem(int codeNumber){
        for (ItemShelf itemShelf : inventory) {
            if (itemShelf.code == codeNumber) {
                itemShelf.setSoldOut(true);
            }
        }
    }
}
```



```
public class ItemShelf {

    int code;
    Item item;
```

```
    boolean soldOut;

    public int getCode() {
        return code;
    }

    public void setCode(int code) {
        this.code = code;
    }

    public Item getItem() {
        return item;
    }

    public void setItem(Item item) {
        this.item = item;
    }

    public boolean isSoldOut() {
        return soldOut;
    }

    public void setSoldOut(boolean soldOut) {
        this.soldOut = soldOut;
    }
}
```

```
public class Item {
    ItemType type;
    int price;

    public ItemType getType() {
        return type;
    }

    public void setType(ItemType type) {
        this.type = type;
    }

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }
}


public enum ItemType {

    COKE,
    PEPSI,
    JUICE,
    SODA;
}
```



```
public class VendingMachineAppDemo {

    public static void main(String args[]){

        VendingMachine vendingMachine = new VendingMachine();
        try {

            System.out.println("|");
            System.out.println("filling up the inventory");
            System.out.println("|");
```

```
11
12              fillUpInventory(vendingMachine);
13              displayInventory(vendingMachine);
14
15              System.out.println("|");
16              System.out.println("clicking on InsertCoinButton");
17              System.out.println("|");
18
19              State vendingState =
    vendingMachine.getVendingMachineState();
20              vendingState.clickOnInsertCoinButton(vendingMachine);
21
22              vendingState = vendingMachine.getVendingMachineState();
23              vendingState.insertCoin(vendingMachine, Coin.NICKEL);
24              vendingState.insertCoin(vendingMachine, Coin.QUARTER);
25              // vendingState.insertCoin(vendingMachine, Coin.NICKEL);
26
27              System.out.println("|");
28              System.out.println("clicking on ProductSelectionButton");
29              System.out.println("|");
30
    vendingState.clickOnStartProductSelectionButton(vendingMachine);
31
32              vendingState = vendingMachine.getVendingMachineState();
33              vendingState.chooseProduct(vendingMachine, 102);
34
35              displayInventory(vendingMachine);
36
37          }
38          catch (Exception e){
39              displayInventory(vendingMachine);
40          }
41
42
43      }
44
45      private static void fillUpInventory(VendingMachine vendingMachine)
    {
46          ItemShelf[] slots =
    vendingMachine.getInventory().getInventory();
47          for (int i = 0; i < slots.length; i++) {
48              Item newItem = new Item();
49              if(i >=0 && i<3) {
50                  newItem.setType(ItemType.COKE);
51                  newItem.setPrice(12);
52              }else if(i >=3 && i<5){
53                  newItem.setType(ItemType.PEPSI);
54                  newItem.setPrice(9);
55              }else if(i >=5 && i<7){
56                  newItem.setType(ItemType.JUICE);
57                  newItem.setPrice(13);
58              }else if(i >=7 && i<10){
59                  newItem.setType(ItemType.SODA);
60                  newItem.setPrice(7);
61              }
62              slots[i].setItem(newItem);
63              slots[i].setSoldOut(false);
64          }
65      }
66
67      private static void displayInventory(VendingMachine
    vendingMachine){
68
69          ItemShelf[] slots =
    vendingMachine.getInventory().getInventory();
70          for (int i = 0; i < slots.length; i++) {
71
72              System.out.println("CodeNumber: " + slots[i].getCode() +
73                      " Item: " + slots[i].getItem().getType().name() +
74                      " Price: " + slots[i].getItem().getPrice() +
75                      " isAvailable: " + !slots[i].isSoldOut());
```

```
76          }
77      }
78
79  }
```

Strategy Pattern

Resources

- Video → 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- Video → 2. Strategy Design Pattern explanation (Hindi) | LLD System Design | Design pattern in Java

Definition

> ***The Strategy pattern is a behavioral design pattern that defines multiple algorithms, encapsulates their logic in dedicated classes, and enables changing an algorithm's behavior at runtime. It's particularly useful when you have multiple ways to perform a task and want to choose the approach dynamically.***

Popular Real-life Examples

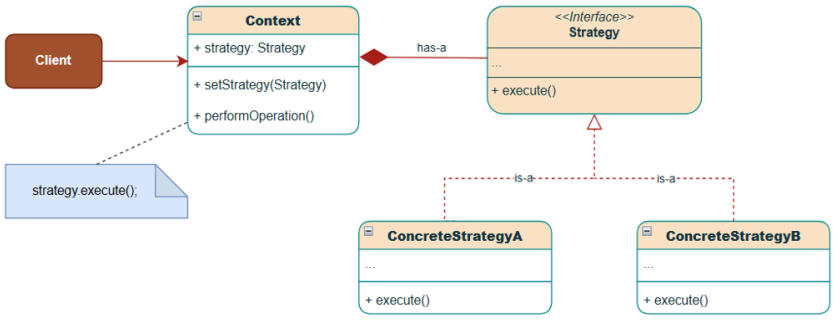
- **Courier Service** → Shipping Cost Calculation - Discount for Premium members, Flat fee, Distance-based based and weight-based computations.
- **Shopping Cart** → Payment Options - CrediCard, PayPal, UPI, Cash, etc.
- **Vehicle Manufacturing** → Different cars(like SUVs, EVs, etc) require different drive modes.

Problems without Strategy Pattern

Refer Strategy Pattern | Implementation section below for a better understanding.

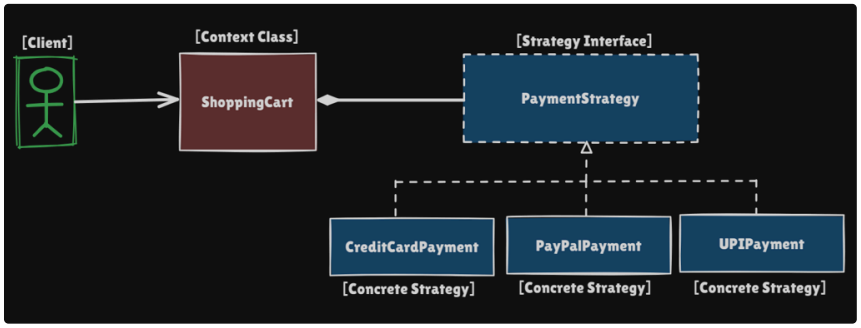
1. Massive Conditional Blocks leading to bloated classes
2. Violation of the Open/Closed Principle and Single Responsibility Violation
3. Code Duplication
4. Tight Coupling
5. Testing Complexity

Class Diagram



Structure of Strategy Pattern

Let's understand the Strategy Pattern using the Payment method example:



- **Strategy Interface(`PaymentStrategy`)**: Defines a common interface(contract) that all concrete strategies must implement.
- **Concrete Strategies(`CreditCardPayment`, `PayPalPayment`, `UPIPayment`)**: Different implementations of the strategy interface, each representing a specific algorithm or approach.
- **Context Class(`ShoppingCart`)**: The class that uses a strategy. It maintains a reference to a strategy object and delegates work to it. Context uses Concrete Strategies to choose the required behavior among the available family of algorithms at runtime.

Implementation

1. Different Drive Modes in a Vehicle

Without Strategy Pattern

```java
public class Vehicle {

    public void drive() {
        System.out.print("\n" + this.getClass().getSimpleName() + ": ");
        System.out.println("Driving Capability: Normal");
    }
}
```

```java
public class SportsVehicle extends Vehicle {

    // Overriding the drive method to provide specific behavior for sports vehicles
    public void drive() {
```

```
        System.out.print("\n" + this.getClass().getSimpleName() + ":
");
        System.out.println("Driving Capability: Sports");
    }
}
```

```
public class OffRoadVehicle extends Vehicle {

    // Overriding the drive method to provide specific behavior
    public void drive() {
        System.out.print("\n" + this.getClass().getSimpleName() + ":
");
        System.out.println("Driving Capability: Sports"); // code
duplication
        // As sports drive mode is not available in the parent class,
we need to override it and implement
        // the specific behavior for all new vehicle types that
require sports drive mode
    }

}
```

```
public class PassengerVehicle extends Vehicle {

    // Reusing the existing drive method from the parent class
    // Driving Capability: Normal
    // No new implementation required
}
```

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Vehicle Drive Modes: Problem Demo");
        Vehicle vehicle = new Vehicle();

        // Sports vehicle - sports drive mode
        vehicle = new SportsVehicle();
        vehicle.drive();

        // Off-road vehicle - sports drive mode
        vehicle = new OffRoadVehicle();
        vehicle.drive();

        // Passenger vehicle - normal drive mode
        vehicle = new PassengerVehicle();
        vehicle.drive();
    }
}
```


With Strategy Pattern

```
// Strategy interface - defines the contract for drive behavior
public interface DriveStrategy {
    public void drive();
}
```

```
// Concrete strategy for normal drive mode
public class NormalDrive implements DriveStrategy {
    @Override
    public void drive() {
        System.out.println("Driving Capability: Normal");
    }
}
// Concrete strategy for sports drive mode
public class SportsDrive implements DriveStrategy {
    @Override
    public void drive() {
        System.out.println("Driving Capability: Sports");
    }
}
// Concrete strategy for electric drive mode
```

```java
public class EVDrive implements DriveStrategy {
    @Override
    public void drive() {
        System.out.println("Driving Capability: Electric");
    }
}
```

```java
// Context class - holds a reference to a strategy object
public class Vehicle {
    DriveStrategy driveStrategy;

    // constructor injection
    public Vehicle(DriveStrategy driveStrategy) {
        this.driveStrategy = driveStrategy;
    }

    public void drive() {
        System.out.print("\n" + this.getClass().getSimpleName() + ":
");
        driveStrategy.drive();
    }
}
// Concrete context subclass
public class GoodsVehicle extends Vehicle {

    public GoodsVehicle(DriveStrategy driveStrategy) {
        super(driveStrategy);
    }
}
// Concrete context subclass
public class SportsVehicle extends Vehicle {

    public SportsVehicle(DriveStrategy driveStrategy) {
        super(driveStrategy);
    }
}
// Concrete context subclass
public class OffRoadVehicle extends Vehicle {

    OffRoadVehicle(DriveStrategy driveStrategy) {
        super(driveStrategy);
    }
}
// Concrete context subclass
public class HybridVehicle extends Vehicle {

    public HybridVehicle(DriveStrategy driveStrategy) {
        super(driveStrategy);
    }
}
```



```java
// Client Code
public class Demo {
    public static void main(String[] args) {
        System.out.println("###### Strategy Design Pattern ######");
        System.out.println("###### Example: Vehicle Drive Modes
######");

        Vehicle vehicle = new SportsVehicle(new SportsDrive());
        vehicle.drive();

        vehicle = new GoodsVehicle(new NormalDrive());
        vehicle.drive();

        vehicle = new HybridVehicle(new EVDrive());
        vehicle.drive();

        vehicle = new GoodsVehicle(new NormalDrive());
        vehicle.drive();
    }
```

```
}
```

2. Shopping Cart Payment Methods

Without Strategy Pattern

```
// A simple payment processor class - bloated with payment logic
public class PaymentProcessor {
    public void processPayment(String type, double amount) {
        switch (type) {
            case "credit_card" -> {
                // x lines of credit card logic
                System.out.println("Paid $" + amount + " using credit card");
            }
            case "paypal" -> {
                // y lines of PayPal logic
                System.out.println("Paid $" + amount + " using PayPal");
            }
            case "net_banking" -> {
                // z lines of bank transfer logic
                System.out.println("Paid $" + amount + " using bank transfer");
            }
            case "cash" -> {
                // 10 lines of cash on delivery logic
                System.out.println("Paid $" + amount + " using cash");
            }
            default -> throw new IllegalStateException("Unexpected value: " + type);
        }
        // Adding another payment method(crypto) requires modifying this class
        // This keeps growing with each new payment method
        // bad design
    }
}
```



```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Payment Processor: Problem Demo");
        PaymentProcessor processor = new PaymentProcessor();
        processor.processPayment("credit_card", 100);
        processor.processPayment("paypal", 200);
        processor.processPayment("net_banking", 300);
        processor.processPayment("cash", 400);
    }
}
```

With Strategy Pattern

```
// Strategy interface
public interface PaymentStrategy {
    void pay(double amount);
}
```

```
// Concrete strategy - for credit card payment
public class CreditCardPayment implements PaymentStrategy {
    private String cardNumber;

    public CreditCardPayment(String cardNumber) {
        this.cardNumber = cardNumber;
    }

    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using credit card ending in "
                + cardNumber.substring(cardNumber.length() - 4));
```

```
    }
}
```

```
// Concrete strategy - for PayPal payment
public class PayPalPayment implements PaymentStrategy {
    private String email;

    public PayPalPayment(String email) {
        this.email = email;
    }

    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using PayPal account " + email);
    }
}
```

```
// Concrete strategy - for UPI payment
public class UPIPayment implements PaymentStrategy {
    private String upiId;

    public UPIPayment(String upiId) {
        this.upiId = upiId;
    }

    public void pay(double amount) {
        System.out.println("Paid $" + amount + " using UPI ID " + upiId);
    }
}
```

```
// Context class - holds reference to a strategy object
public class ShoppingCart {
    private PaymentStrategy paymentStrategy;

    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.paymentStrategy = strategy;
    }

    public void checkout(double amount) {

System.out.print(this.paymentStrategy.getClass().getSimpleName() + ": ");
        paymentStrategy.pay(amount);
    }
}
```



```
// Client code - to simulate payment processing
public class Demo {
    public static void main(String[] args) {
        System.out.println("###### Strategy Design Pattern ######");
        System.out.println("###### Example: Payment Processor ######");

        // Create a shopping cart and set payment strategy
        ShoppingCart cart = new ShoppingCart();

        // Choosing payment behavior at runtime
        cart.setPaymentStrategy(new CreditCardPayment("1234-5678-9012-3456"));
        cart.checkout(100.0);
        cart.setPaymentStrategy(new PayPalPayment("johndoe@example.com"));
        cart.checkout(200.0);
        cart.setPaymentStrategy(new UPIPayment("9988776655@ybl"));
        cart.checkout(300.0);
        // Adding another payment method(crypto) is as simple as adding a new strategy class
        // No need to modify existing code - good design
        // cart.setPaymentStrategy(new CryptoPayment("BTC"));
```

```
        // cart.checkout(400.0);
    }
}
```

Template Method Pattern

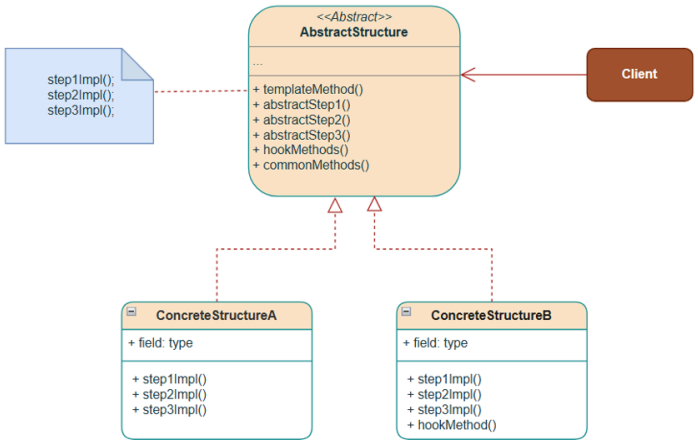
Resources

- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 39. Template Method Design Pattern Explanation in Java | Concept and Coding LLD | Low Level Design

Definition

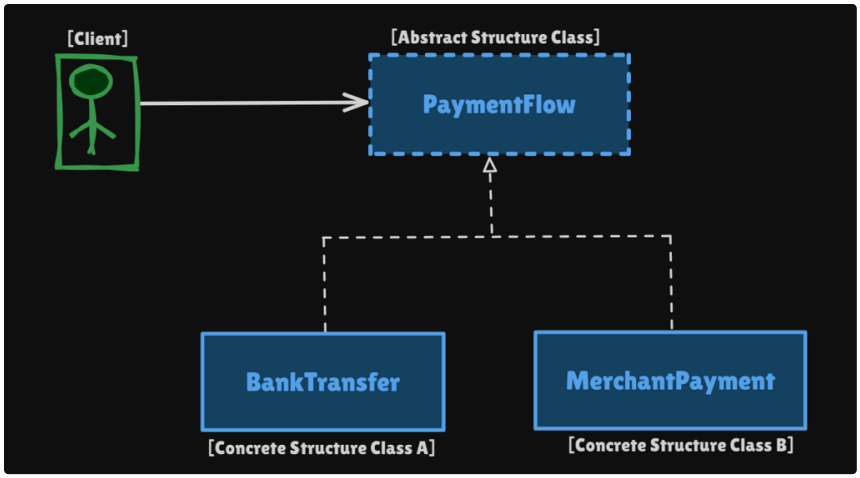
> **The Template Method pattern is a behavioral design pattern that defines the skeleton/structure of an algorithm(common workflow) in a base class and allows subclasses to override specific steps and provide custom implementation without changing the algorithm's core workflow.**

Class Diagram



Structure of the Template Method Pattern

Let's understand the Structure of the Template Method Pattern using the Payment Workflows example:



1. **Abstract Structure Class (e.g., `PaymentFlow`):**
   - Has a template method: Defines the sequence of steps for the algorithm (the skeleton of the payment processing workflow). Calls both abstract methods and hook methods in a specific sequence
   - Declares one or more abstract methods that must be implemented by each subclass. These define the varying parts(specific implementation) of the algorithm.
   - May contain hook methods: have default implementations. Subclasses can choose to override these for further customization if needed.
   - May also contain common methods: They are implemented once in the base class, and all subclasses share this common functionality.
2. **Concrete Structure Classes** (e.g., `BankTransfer`, `MerchantPayment`)
   - These classes extend the abstract base class and provide specific implementations for the abstract methods defined in the template.
   - They also optionally override hook methods to provide customization of business workflows. Improves flexibility.

Implementation(Example: Payment Workflows)

```java
// Abstract class
public abstract class PaymentFlow {

    // Abstract methods - these methods are implemented by the subclasses.
    public abstract void validateRequest();

    public abstract void debitAmount();

    public abstract void calculateFees();

    public abstract void creditAmount();

    // Template method: which defines the order of steps to execute the task.
    public final void sendMoney() {
        // step 1
        validateRequest();
        // step 2
        debitAmount();
        // step 3
        calculateFees();
        // step 4
        creditAmount();
    }

```

```
    // Hook method: which can be overridden by the subclasses.
    protected boolean requiresOTPAuthentication() {
        return false; // Default: authentication not required
    }

    // Common method: All subclasses share this common functionality.
    public void logTransaction() {
        System.out.println("Transaction Completed!");
    }
}
```

```
// Concrete class
public class BankTransfer extends PaymentFlow {
    @Override
    public void validateRequest() {
        System.out.println("[+] Specific Validation Logic for Bank
Transfer");
    }

    @Override
    public void debitAmount() {
        System.out.println("[+] Specific Debit Amount Logic for Bank
Transfer");
    }

    @Override
    public void calculateFees() {
        System.out.println("[+] Specific Fee Calculation Logic for
Bank Transfer. 0% Fees is applied.");
    }

    @Override
    public void creditAmount() {
        System.out.println("[+] Specific Credit Amount Logic for Bank
Transfer. Full amount is credited.");
    }

}
```



```
// Concrete class
public class MerchantPayment extends PaymentFlow {
    @Override
    public void validateRequest() {
        System.out.println("[+] Specific Validation Logic for Merchant
Payment");
    }

    @Override
    public void debitAmount() {
        if (requiresOTPAuthentication()) {
            System.out.println("[+] Perform OTP Authentication.");
        }
        System.out.println("[+] Specific Debit Amount Logic for
Merchant Payment");
    }

    @Override
    public void calculateFees() {
        System.out.println("[+] Specific Fee Calculation Logic for
Merchant Payment. 2% Fees is applied.");
    }

    @Override
    public void creditAmount() {
        System.out.println("[+] Specific Credit Amount Logic for
Merchant Payment. Remaining amount is credited.");
    }

    // Hook method - overridden by subclass
    @Override
    protected boolean requiresOTPAuthentication() {
```

```
        return true;
    }
}
```

```
// Client class
public class TemplateDemo {
    public static void main(String[] args) {
        System.out.println("###### Template Method Design Pattern ######");

        // Bank Transfer
        System.out.println("===== Bank Transfer =====");
        PaymentFlow bankTransfer = new BankTransfer();
        bankTransfer.sendMoney(); // Template method
        bankTransfer.logTransaction(); // Common method

        // Merchant Payment
        System.out.println("===== Merchant Payment =====");
        PaymentFlow merchantPayment = new MerchantPayment();
        merchantPayment.sendMoney(); // Template method
        merchantPayment.logTransaction(); // Common method
    }
}
```

Output

```
###### Template Method Design Pattern ######
===== Bank Transfer =====
[+] Specific Validation Logic for Bank Transfer
[+] Specific Debit Amount Logic for Bank Transfer
[+] Specific Fee Calculation Logic for Bank Transfer. 0% Fees is applied.
[+] Specific Credit Amount Logic for Bank Transfer. Full amount is credited.
Transaction Completed!
===== Merchant Payment =====
[+] Specific Validation Logic for Merchant Payment
[+] Perform OTP Authentication.
[+] Specific Debit Amount Logic for Merchant Payment
[+] Specific Fee Calculation Logic for Merchant Payment. 2% Fees is applied.
[+] Specific Credit Amount Logic for Merchant Payment. Remaining amount is credited.
Transaction Completed!

Process finished with exit code 0
```

Visitor Pattern

Resources

- 41. All Behavioral Design Patterns | Strategy, Observer, State, Template, Command, Visitor, Memento
- 36. Visitor Design Pattern | Double Dispatch | Low Level Design

Definition

> **The Visitor design pattern is a behavioral design pattern that allows you to add new operations to existing object structures without modifying those structures. It achieves this by separating the algorithm from the actual objects that it operates on. It uses Double Dispatch to implement this.**

Double Dispatch is a method where the runtime type of the caller object (the receiver) and the runtime type of the argument object (the visitor) determine which method is invoked.

In simple terms, Double Dispatch is the selection of a method based on the runtime types of two objects, the receiver and the argument.

More on this later...

The Problem: Naive Approach

```java
// Bloated Element class with multiple operations
public class SuiteHotelRoom {
    private String roomNumber;
    private String numberOfRooms;

    public SuiteHotelRoom(String roomNumber, String numberOfRooms) {
        this.roomNumber = roomNumber;
        this.numberOfRooms = numberOfRooms;
    }

    public void clean() {
        System.out.println("Housekeeping: Cleaning suite " +
                roomNumber + " with " +
                numberOfRooms + " rooms (90 minutes)");
    }

    public void deliverRoomService(String orderDetails) {
        System.out.println("Room Service: VIP delivery of " +
orderDetails +
```

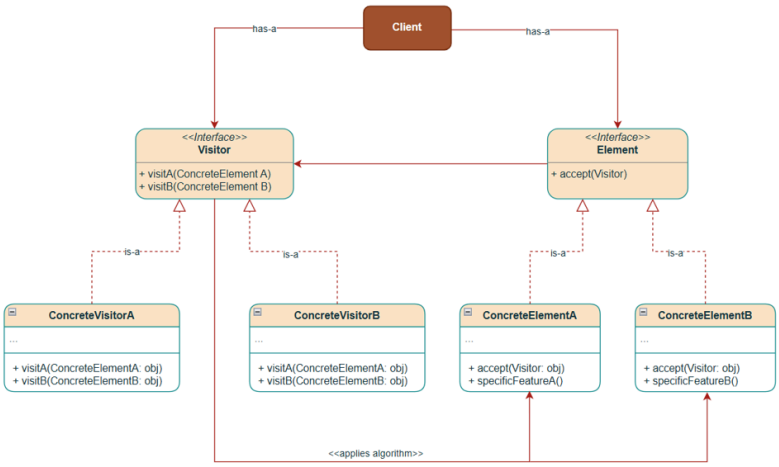
```
                " to suite " + roomNumber +
                " with full dining setup");
    }

    public double calculatePrice() {
        System.out.println("Pricing: Suite " + roomNumber +
                " - Rs. 2000/night");
        return 500.0;
    }

    // many more operations can come over time
}
```

```
// Usage
public class Demo {
    public static void main(String[] args) {
        System.out.println("##### Visitor Pattern: Problem Demo
#####");
        SuiteHotelRoom suite = new SuiteHotelRoom("301", "2");
        suite.clean();
        suite.deliverRoomService("Breakfast");
        suite.calculatePrice();
    }
}
```

The Key Problems Without Visitor Pattern

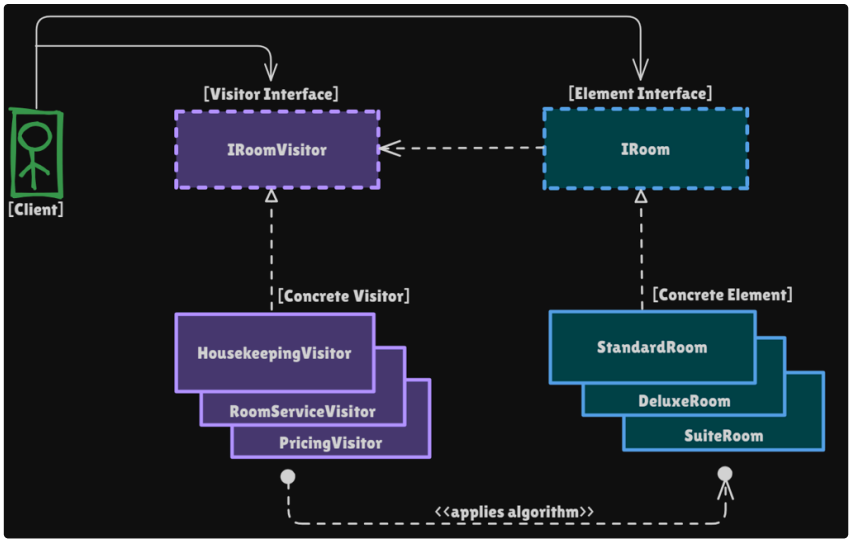
1. Violates the Open/Closed Principle and the Single Responsibility Principle, making it unextensible.
2. Scattered Operation Logic leading to redundant code.
3. Difficult Testing
4. Tight coupling & Poor Reusability of code

Class Diagram



Structure of Visitor Pattern

Understanding the Structure of the Visitor Pattern using the Hotel Room example:



- **Element Interface(** `IRoom` **)**: Declares a contract for all concrete element types to implement the `accept(IRoomVisitor visitor)` method, which allows visitors to perform operations on them.
- **Concrete Element(** `StandardRoom` , `DeluxeRoom` , and `SuiteRoom` **)**: These concrete element types are actual objects on which the algorithm is applied. Inside the `accept(IRoomVisitor visitor)` method, they call the appropriate visitor method for their respective types.
- **Visitor Interface(** `IRoomVisitor` **)**: Defines one `visit(ConcreteElement e)` method for each room type. This is the core logic of the pattern. The algorithm is executed on the object.
- **Concrete Visitor(** `HousekeepingVisitor` , `RoomServiceVisitor` , `PricingVisitor` **)**: Each visitor service implements how to handle the particular operation on each element type differently.

Implementation

```java
// Visitor interface - defines operations
public interface IRoomVisitor {
    void visitStandardRoom(StandardRoom room);

    void visitDeluxeRoom(DeluxeRoom room);

    void visitSuiteRoom(SuiteRoom room);
}
```

```java
// Element interface - represents rooms(elements) that can be visited
public interface IRoom {
    void accept(IRoomVisitor visitor);

}
```

```java
// Concrete Element - Different room types
public class StandardRoom implements IRoom {
    private final String roomNumber;

    public StandardRoom(String roomNumber) {
        this.roomNumber = roomNumber;
    }

    @Override
    public void accept(IRoomVisitor visitor) {
        visitor.visitStandardRoom(this);
    }

    public String getRoomNumber() {
```

```
        return roomNumber;
    }
}
```

```
// Concrete Element - Different room types
public class DeluxeRoom implements IRoom {
    private final String roomNumber;
    private final boolean hasJacuzzi;

    public DeluxeRoom(String roomNumber, boolean hasJacuzzi) {
        this.roomNumber = roomNumber;
        this.hasJacuzzi = hasJacuzzi;
    }

    @Override
    public void accept(IRoomVisitor visitor) {
        visitor.visitDeluxeRoom(this);
    }

    public String getRoomNumber() {
        return roomNumber;
    }

    public boolean hasJacuzzi() {
        return hasJacuzzi;
    }
}
```

```
// Concrete Element - Different room types
public class SuiteRoom implements IRoom {
    private final String roomNumber;
    private final int numberOfRooms;

    public SuiteRoom(String roomNumber, int numberOfRooms) {
        this.roomNumber = roomNumber;
        this.numberOfRooms = numberOfRooms;
    }

    @Override
    public void accept(IRoomVisitor visitor) {
        visitor.visitSuiteRoom(this);
    }

    public String getRoomNumber() {
        return roomNumber;
    }

    public int getNumberOfRooms() {
        return numberOfRooms;
    }
}
```



```
// Concrete Element - Different room types
public class SuiteRoom implements IRoom {
    private final String roomNumber;
    private final int numberOfRooms;

    public SuiteRoom(String roomNumber, int numberOfRooms) {
        this.roomNumber = roomNumber;
        this.numberOfRooms = numberOfRooms;
    }

    @Override
    public void accept(IRoomVisitor visitor) {
        visitor.visitSuiteRoom(this);
    }

    public String getRoomNumber() {
        return roomNumber;
    }

```

```
    public int getNumberOfRooms() {
        return numberOfRooms;
    }
}
```

```
// Concrete Element - Different room types
public class SuiteRoom implements IRoom {
    private final String roomNumber;
    private final int numberOfRooms;

    public SuiteRoom(String roomNumber, int numberOfRooms) {
        this.roomNumber = roomNumber;
        this.numberOfRooms = numberOfRooms;
    }

    @Override
    public void accept(IRoomVisitor visitor) {
        visitor.visitSuiteRoom(this);
    }

    public String getRoomNumber() {
        return roomNumber;
    }

    public int getNumberOfRooms() {
        return numberOfRooms;
    }
}
```


```
// Concrete Element - Different room types
public class SuiteRoom implements IRoom {
    private final String roomNumber;
    private final int numberOfRooms;

    public SuiteRoom(String roomNumber, int numberOfRooms) {
        this.roomNumber = roomNumber;
        this.numberOfRooms = numberOfRooms;
    }

    @Override
    public void accept(IRoomVisitor visitor) {
        visitor.visitSuiteRoom(this);
    }

    public String getRoomNumber() {
        return roomNumber;
    }

    public int getNumberOfRooms() {
        return numberOfRooms;
    }
}
```



```
// Concrete Visitor - demonstrates adding new operations easily
public class HousekeepingVisitor implements IRoomVisitor {
    @Override
    public void visitStandardRoom(StandardRoom room) {
        System.out.println("Housekeeping: Cleaning standard room " +
                room.getRoomNumber() + " (30 minutes)");
    }

    @Override
    public void visitDeluxeRoom(DeluxeRoom room) {
        System.out.println("Housekeeping: Cleaning deluxe room " +
                room.getRoomNumber() +
                (room.hasJacuzzi() ? " including jacuzzi" : "") +
                " (45 minutes)");
    }

    @Override
    public void visitSuiteRoom(SuiteRoom room) {
```

```
        System.out.println("Housekeeping: Cleaning suite " +
                room.getRoomNumber() + " with " +
                room.getNumberOfRooms() + " rooms (90 minutes)");
    }
}
```

```
// Pricing visitor - demonstrates adding new operations easily
public class RoomServiceVisitor implements IRoomVisitor {
    private final String orderDetails;

    public RoomServiceVisitor(String orderDetails) {
        this.orderDetails = orderDetails;
    }

    @Override
    public void visitStandardRoom(StandardRoom room) {
        System.out.println("Room Service: Delivering " + orderDetails
+
                " to standard room " + room.getRoomNumber());
    }

    @Override
    public void visitDeluxeRoom(DeluxeRoom room) {
        System.out.println("Room Service: Premium delivery of " +
orderDetails +
                " to deluxe room " + room.getRoomNumber() +
                " with complimentary champagne");
    }

    @Override
    public void visitSuiteRoom(SuiteRoom room) {
        System.out.println("Room Service: VIP delivery of " +
orderDetails +
                " to suite " + room.getRoomNumber() +
                " with full dining setup");
    }
}
```



```
// Pricing visitor - demonstrates adding new operations easily
public class PricingVisitor implements IRoomVisitor {
    private double totalRevenue = 0;

    @Override
    public void visitStandardRoom(StandardRoom room) {
        double price = 1000.0;
        totalRevenue += price;
        System.out.println("Pricing: Standard room " +
room.getRoomNumber() +
                " - Rs." + price + "/night");
    }

    @Override
    public void visitDeluxeRoom(DeluxeRoom room) {
        double price = 2000.0;
        totalRevenue += price;
        System.out.println("Pricing: Deluxe room " +
room.getRoomNumber() +
                " - Rs." + price + "/night");
    }

    @Override
    public void visitSuiteRoom(SuiteRoom room) {
        double price = 5000.0;
        totalRevenue += price;
        System.out.println("Pricing: Suite " + room.getRoomNumber() +
                " - Rs." + price + "/night");
    }

    public double getTotalRevenue() {
        return totalRevenue;
    }
```

```
}
```

```
// Usage
public class HotelVisitorDemo {
    public static void main(String[] args) {
        System.out.println("\n###### Visitor Design Pattern Demo ######");

        // Create different room types(elements) - Standard, Deluxe, Suite
        IRoom[] rooms = {
                new StandardRoom("101"),
                new DeluxeRoom("201", true),
                new SuiteRoom("301", 3),
                new StandardRoom("102"),
                new DeluxeRoom("202", false)
        };

        // Calling Visitors on elements
        System.out.println("\n==> Housekeeping Service");
        IRoomVisitor housekeeping = new HousekeepingVisitor();
        for (IRoom room : rooms) {
            room.accept(housekeeping);
        }

        System.out.println("\n==> Room Service");
        IRoomVisitor roomService = new RoomServiceVisitor("Breakfast");
        rooms[0].accept(roomService);  // Deliver to standard room
        rooms[1].accept(roomService);  // Deliver to deluxe room
        rooms[2].accept(roomService);  // Deliver to suite

        System.out.println("\n==> Revenue Calculation");
        PricingVisitor pricing = new PricingVisitor();
        for (IRoom room : rooms) {
            room.accept(pricing);
        }
        System.out.println("Total Revenue: Rs." + pricing.getTotalRevenue());
    }
}
```


Output

```
###### Visitor Design Pattern Demo ######

==> Housekeeping Service
Housekeeping: Cleaning standard room 101 (30 minutes)
Housekeeping: Cleaning deluxe room 201 including jacuzzi (45 minutes)
Housekeeping: Cleaning suite 301 with 3 rooms (90 minutes)
Housekeeping: Cleaning standard room 102 (30 minutes)
Housekeeping: Cleaning deluxe room 202 (45 minutes)

==> Room Service
Room Service: Delivering Breakfast to standard room 101
Room Service: Premium delivery of Breakfast to deluxe room 201 with complimentary champagne
Room Service: VIP delivery of Breakfast to suite 301 with full dining setup

==> Revenue Calculation
Pricing: Standard room 101 - Rs.1000.0/night
Pricing: Deluxe room 201 - Rs.2000.0/night
Pricing: Suite 301 - Rs.5000.0/night
Pricing: Standard room 102 - Rs.1000.0/night
Pricing: Deluxe room 202 - Rs.2000.0/night
Total Revenue: Rs.11000.0

Process finished with exit code 0
```

How does the Visitor Pattern achieve Double Dispatch?

Single Dispatch

Single dispatch means the method that gets executed is determined by the runtime type of only one object, typically the receiver of the method call.

```
// No visitors

// Just different types of rooms - IRoom - Standard, Deluxe, Suite
// each implementing its own version of accept

IRoom roomObj = new DeluxeRoom();
roomObj.accept();

// At runtime we execute accept() method implementation of DeluxeRoom class
// The method executed depends  only on the runtime type of roomObj
// (which is DeluxeRoom), not on any other factors.
```

Double Dispatch

Double Dispatch comes into the picture when you need behavior, i.e., actual method execution, to depend on multiple object types:

1. The runtime type of the caller object (i.e., the receiver or element) → first dispatch
2. The runtime type of the object passed as an argument (i.e., the visitor) → second dispatch

```
// Different types of Visitors - IRoomVisitor - HousekeepingVisitor,
// RoomServiceVisitor and PricingVisitor

// Different types of rooms - IRoom - Standard, Deluxe, Suite
IRoom myroom = new StandardRoom("101");

IRoomVisitor myvisitor = new HousekeepingVisitor();

myroom.accept(myvisitor); // DOUBLE DISPATCH

// The runtime type of the caller object - myroom is resolved at runtime &
// StandardRoom.accept() is called ---> FIRST DISPATCH

// The runtime type of the object passed as an argument - inside the accept() method
// myvisitor is resolved at runtime & HousekeepingVisitor.visitStandardRoom()
// is called ---> SECOND DISPATCH
```


Visitor vs Strategy Design Pattern

Strategy Pattern: Encapsulates **alternative algorithms** for a single operation.

- Allows swapping different behaviors at runtime.
- These algorithms are independent of the object they are applied.
- Here, we find Different ways to do ONE thing.

Visitor Pattern: Adds **new operations** to an object structure without modifying it.

- Allows adding operations across multiple types.
- The new operations are tied to specific objects they are applied.
- Here we perform Different operations on MANY types.